

StarfishOS: Revisiting Single System Image on CXL with State-Partitioned Microkernel

Fangnuo Wu, Jingsheng Yan, Mingkai Dong, Wenjun Cai, Jingwei Xu, Tong Xin, Haibo Chen
Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University
Institute of Artificial Intelligence Operating Systems, Shanghai Jiao Tong University
Engineering Research Center for Domain-specific Operating Systems, Ministry of Education, China

Abstract

Single System Image (SSI) offers an attractive OS abstraction that enables single-machine applications to transparently execute on multiple machines with improved resource utilization. The emergence of Compute Express Link (CXL) shifts cross-machine synchronization from message passing to coherent shared memory, which may help overcome SSI's long-standing adoption barrier caused by inefficient cross-machine message passing. However, placing state in slow CXL memory degrades performance and makes the system vulnerable to partial failures, since inconsistent CXL-resident state may block or crash the whole system.

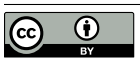
This paper notes that microkernels' minimized kernel-space state and capability-based state management allow easier state partitioning to minimize state on CXL. To this end, we propose a *state-partitioned microkernel*, and realize it in StarfishOS, the first OS-level SSI that leverages CXL shared memory for both high efficiency and partial failure resilience by keeping CXL state minimal and recoverable. Evaluations show that applications can transparently scale on StarfishOS with zero code changes, and achieve up to 5.2× and 4.1× the throughput of their distributed and CXL-optimized versions, respectively. Moreover, StarfishOS utilizes all stranded CPU resources by running applications across machines.

StarfishOS is open source and available at https://github.com/starfishos-org/starfishos.

CCS Concepts: • **Software and its engineering** → **Operating systems**; • **Computer systems organization** → **Reliability**.

Keywords: Single System Image (SSI), Microkernel, Compute Express Link (CXL), Partial Failure Resilience

ACM Reference Format:
Fangnuo Wu, Jingsheng Yan, Mingkai Dong, Wenjun Cai, Jingwei Xu, Tong Xin, Haibo Chen. 2026. StarfishOS: Revisiting Single System Image on CXL with State-Partitioned Microkernel. In *ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP '26), September 29-October 02, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 22 pages. https://doi.org/10.1145/3830418.3843894



This work is licensed under a Creative Commons Attribution 4.0 International License.
SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/2026/09
https://doi.org/10.1145/3830418.3843894

1 Introduction

Compute Express Link (CXL) [3] connects rack-scale machines into a *CXL pod* [74], where all machines access a shared memory pool through load/store [4]. This shared-memory primitive replaces inter-machine messaging with load/store access, enabling much cheaper cross-machine synchronization. Prior work has already demonstrated the potential of this model. For example, CXL-SHM [80] builds a MapReduce program from scratch to exchange shuffle data through CXL shared memory to reduce execution time. Tigon [40], on the other hand, reorganizes database data structures to be CXL-aware, avoiding two-phase commit [34] and improving OLTP throughput by 18.5×.

Despite their performance gains, a broader question arises: to benefit from CXL, is it necessary to re-design and re-implement every application, especially those with large and hard-to-modify codebases?

To answer this question, we revisit OS-level single system image (SSI) [27, 30, 68, 73] as a way to enable existing single-machine shared-memory applications to transparently benefit from CXL. SSI exposes a CXL pod as one logical machine with a single address space, PID namespace, file namespace, and device namespace. Under SSI, the OS transparently schedules application threads pod-wide and routes all cross-machine operations through CXL load/store, without any application redesign.

Building SSI on CXL, however, is non-trivial. At one extreme, the system can keep all OS and application state in local DRAM and communicate across machines via message passing, as traditional SSI systems do [27, 30, 73]. However, this design fails to exploit CXL's shared memory semantics and remains inefficient due to message-passing [25, 46, 52, 69]. At the other extreme, the system can place all OS and application state on CXL, so the pod behaves like a shared-memory machine on which existing OSes and applications run unmodified. While this design is simple and fully exploits CXL's shared-memory semantics, it faces two obstacles:

(1) *Efficiency*: CXL is slower than local DRAM: 2–5× access latency and 20%–80% bandwidth of local DRAM [62, 70, 80].

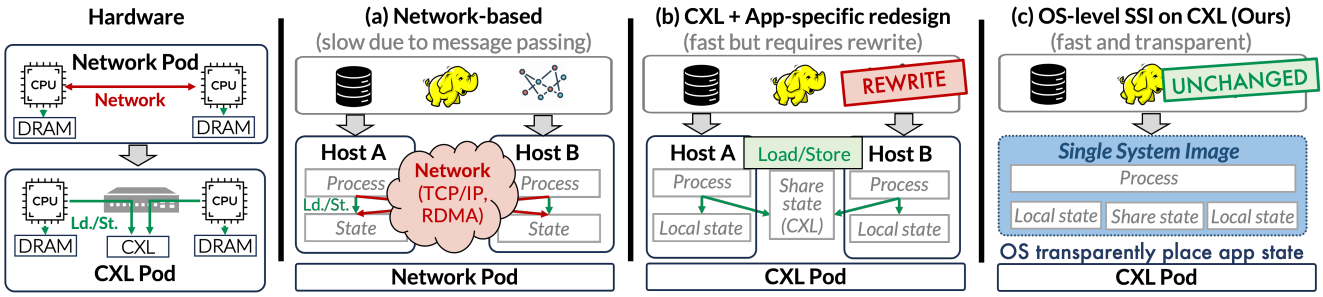


Figure 1. Comparison of our OS-level SSI approach with network-based and application-specific redesign approaches. *For many applications, network-based message passing (a) introduces extra overheads of synchronization, serialization, and copying. Some works redesign applications (b) to leverage CXL to eliminate these overheads, and our OS-level SSI (c) achieves the same benefits transparently.*

Placing all kernel and application state in CXL causes unacceptable overhead for kernel and applications (up to 2.4× slowdown on our platform, as shown in Figure 14).

(2) *Partial failure*: in a CXL pod, host machines and CXL memory modules have separate power supplies and can fail independently [80, 84]. A host crash may leave CXL-resident locks unreleased or leave data structures in inconsistent states; surviving machines that observe them may block or crash, turning a single-host failure into a pod-wide outage.

Key insight. Both obstacles stem from the same cause: placing too much OS state on CXL. *State partitioning* resolves both simultaneously, i.e., keeping only the minimal coordination state in CXL, and leaving everything else in private DRAM for efficiency. Because this shared state is small, it can be engineered for partial-failure resilience, so a host crash need not block or crash the surviving machines.

Realizing this in practice requires a *clear state-management abstraction*: the OS must explicitly identify each state object (a typed unit of OS-managed state) and each dependency edge (an explicit reference through which one object can reach another). Monolithic kernels generally lack such an explicit model—their state is spread across tightly coupled kernel subsystems with implicit references—so fine-grained partitioning and placement become impractical [38, 66]. Microkernels, in contrast, are suited for state partitioning. First, their kernel-space state is small, enumerable, and individually manageable [42, 79]. Second, microkernels' capability-based state management tracks each OS object as a first-class entity with explicit type and capabilities for access control, enabling per-object placement decisions [42, 79].

Contributions. We present StarfishOS, the first OS-level SSI for CXL pods with a *state-partitioned microkernel*. Each layer of the OS stack, including kernel-space modules (IPC, scheduling, memory management), system services (filesystems, process managers, etc.), and applications, faces a distinct state-sharing challenge; StarfishOS addresses each with a tailored partitioning and OS-level resilience strategy.

For efficiency, StarfishOS applies the same partitioning rule across the OS stack: only state needed for cross-machine coordination resides in CXL, while locality-sensitive state remains in private DRAM. In the kernel, this means sharing only the data structures that implement inter-machine functionality, such as IPC queues, scheduling queues, and the CXL allocator. In system services, StarfishOS designs collaborative system service instances to keep only global coordination metadata in CXL and leave service state inside per-machine service instances. For applications, StarfishOS uses OS-visible semantics to partition application state into objects and place each object in either CXL or private DRAM. StarfishOS also introduces a page-fault-driven mechanism that migrates heap pages to CXL only on demand. This makes CXL usage small and aligned with actual state sharing rather than with putting everything on CXL.

For handling partial failures, StarfishOS targets OS resilience rather than transparent application recovery. A machine crash should therefore resemble the loss of processes on a single machine, without blocking or corrupting the rest of the pod. To achieve this, StarfishOS makes its CXL-resident kernel coordination state recoverable with lock-free, detectably recoverable data structures [19, 33], keeps most service state outside CXL so failed service instances can be re-created via micro-reboot, and leaves application-level recovery to the applications' own mechanisms. As a result, surviving machines can continue using both the OS and the shared CXL state after other machines fail.

Evaluation. We evaluate StarfishOS's two main benefits for existing single-machine applications. First, StarfishOS transparently scales unmodified shared-memory applications to the full pod: applications such as Phoenix [55], DB×1000 [6], and GeminiGraph [83] scale to 8 machines with 2.0–3.5× speedup and zero code changes. For Phoenix and GeminiGraph, StarfishOS outperforms their distributed-style baselines by 1.7× and 5.2×, respectively. For DB×1000, StarfishOS achieves 4.1× speedup compared to Tigon [40].

Second, StarfishOS enables applications to utilize all stranded CPU resources by running applications across machines. In traditional OSes, a multithreaded application must be bin-packed onto one machine, so fragmented CPUs across machines cannot be used. In contrast, StarfishOS lets applications use all idle CPUs across machines.

In summary, this paper makes the following contributions:

- We revisit SSI in the CXL era and present the first OS-level SSI that uses CXL's shared-memory primitive for cross-machine synchronization.
- We propose the state-partitioned microkernel to achieve efficient SSI with OS-level partial-failure resilience.
- We support automatic scaling of existing single-machine applications across multiple machines and improve pod-wide resource utilization.

2 Revisiting Single System Image with CXL

CXL can yield application performance gains. Compute Express Link (CXL) [3] enables multiple hosts to access a shared memory pool using load and store instructions. In a CXL pod (Figure 1 (b)), CXL allows machines to coordinate through shared-memory operations, reducing the communication overhead of two-phase commit (2PC) in distributed transactions [34] and of MapReduce shuffling [5].

Current gains often require application-specific redesign. So far, the large performance gains often rely on *application-specific redesign*. For example, Tigon [40] introduces more than 5,000 lines of changes to an 18,000-line codebase to restructure database data structures, and manage data placement between local DRAM and shared CXL memory. Despite this effort, Tigon remains specialized to locking-based concurrency control, including 2PL [34] and next-key locking [34]. CXL-SHM [80] exhibits a similar pattern: although it reports comparable gains, it rewrites MapReduce from scratch to accommodate its pass-by-reference design.

Our choice: OS-level SSI.. We therefore ask whether applications can gain CXL's benefits *without per-application redesign*, and whether a single approach can apply to a *broad class* of applications. Since decades of experience show that single-machine, multithreaded applications already exploit shared-memory semantics effectively, we explore a different path: an *OS-level single system image (SSI)* that makes a CXL pod appear as a large shared-memory machine. SSI brings two direct advantages:

(1) Auto-scaling existing programs to transparently benefit from CXL: legacy single-machine applications, whether single- or multi-process, can be auto-scaled to run on any CPU in the CXL pod with zero code changes. As shown in Figure 1 (c), the OS transparently schedules a process's threads across the pod and places its state on DRAM/CXL memory on demand, so cross-node communication transparently becomes load/store rather than message passing.

(2) Better resource utilization and failure handling: with a single view of pod-wide resources provided by SSI, applications can transparently use remote devices such as SSDs/NICs. Prior work reports that pooling devices across 8 hosts can increase device utilization by 46–81% [82]. Applications can fail over to remote devices when local ones fail, avoiding repair downtime [81, 82].

SSI is appealing in cloud environments. A tenant may submit a legacy in-memory analytics job written for a single machine, but the provider may have only fragmented resources across a CXL pod rather than one sufficiently large machine. Without SSI, the tenant must wait for a sufficiently large server or rewrite the application for distributed or CXL-aware execution. With OS-level SSI, the provider can aggregate pod-wide resources and run the job transparently as one larger machine, without application changes.

Why not choose user-level or hypervisor-level SSI?. We consider three possible layers for realizing SSI on a CXL pod: (1) user-level SSI, i.e., distributed shared memory (DSM) libraries [18, 25, 46, 48, 52, 69], (2) hypervisor-level SSI [28, 59, 63], and (3) OS-level SSI [20, 27, 30, 49, 68, 73]. We choose OS-level SSI because it is the only approach that can *both* transparently run existing shared-memory applications and globally manage pod-wide resources.

User-level SSI, i.e., DSM libraries [18, 25, 46, 48, 52, 69], also lets programmers write as if on a single machine, but CXL-based DSMs still require *application-specific redesign* for efficient DRAM/CXL data placement. They also cannot provide pod-wide resource pooling that we target.

Hypervisor-level SSI [28, 59, 63] uses a distributed hypervisor to aggregate rack-scale resources. However, enabling applications to run correctly and efficiently on a CXL pod still requires substantial guest-OS changes to partition guest-OS's state. For SSI, implementing the system at the OS level is more effective than doing so at the hypervisor level, because the OS has richer semantic visibility into OS components, and the hypervisor sees guest memory only at page granularity and cannot distinguish different types of guest memory regions (e.g., heap, stack, and kernel data structures).

3 State-Partitioned Microkernel

3.1 Challenges of building SSI atop CXL

Building OS-level SSI atop CXL requires both efficiency and robustness: it should use shared memory for cross-machine synchronization while *partitioning system state* to minimize the amount of state placed in CXL, and it should tolerate partial machine failures so that partially updated shared state does not block or crash surviving machines.

3.2 Why microkernels are a good fit

We find that microkernel architecture is a good fit for building SSI atop CXL for two main reasons:

(1) Decoupled and Structured State Organization: A microkernel gives system state a clearer organization than a monolithic kernel. At a coarse granularity, it *decouples* kernel state from service state: the kernel retains only a small amount of state for basic mechanisms (e.g., IPC and scheduling), while most OS services, including file systems and device drivers, run in user space with their own state. As a result, major categories of system state are placed behind

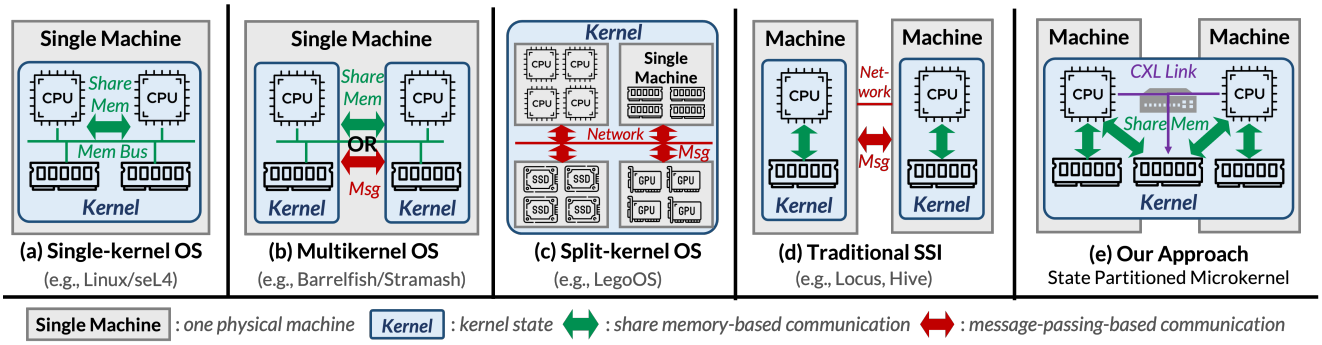


Figure 2. Kernel architecture of existing OSes and our proposed state-partitioned microkernel.

explicit boundaries, rather than being mixed inside the kernel space like in monolithic kernels.

At a finer granularity, capability-based microkernels organize state through capabilities [36] in a *structured* way, which represents system state as objects with explicit relations among them [42, 61, 76, 79]. For example, each process has a `cap_group` object, which tracks all objects that the process can access. This gives system state a finer-grained and structured organization, making dependencies easier to reason about. Thus, state partition can be performed at the object level. By contrast, although monolithic kernels also maintain kernel objects (e.g., `task_struct`, `mm_struct`, `signal_struct`), these objects are often spread across kernel space and connected through implicit dependencies, making system state harder to isolate and partition correctly.

(2) Easier service recovery: microkernels can recover a failed user-space system service by micro-rebooting it [26], i.e., restarting only the failed service instance without rebooting the whole OS. Prior work like Ananke [50] has proposed a detailed micro-rebooting mechanism for a filesystem service. In contrast, in monolithic kernels, filesystems and many other services run inside the kernel and share kernel state with the rest of the system, making isolated restart much harder since the OS cannot easily restart kernel-space system services on other machines. Micro-rebooting is especially useful under partial failures, because if one machine fails and takes the service down with it, the service can be restarted on another machine without rebooting the whole OS.

3.3 State-partitioned microkernel

Based on microkernel's advantages of decoupled and structured state organization and easier service recovery, we propose a *state-partitioned microkernel architecture* for SSI atop CXL. It explicitly divides SSI state into three layers: (1) kernel-space modules, (2) user-space system services, and (3) applications. This structure lets us use layer-specific state-partitioning and partial-failure-handling mechanisms as summarized in Table 1.

Note that we do not claim that microkernel is a necessary condition for state partitioning. Instead, the above properties make microkernel a practical engineering foundation for building SSI atop CXL. A monolithic kernel can apply the same partitioning rule by explicitly identifying all objects and their dependencies and providing isolated service recovery.

3.4 Differences from Existing OSes

As shown in Figure 2, our proposed state-partitioned microkernel departs from existing OSes in three main aspects:

(1) Cross-machine coordination with efficient shared memory instead of message passing. Single-machine OSes (Figure 2(a)) assume only local memory and do not support cross-machine coordination. Multikernel OSes such as Barrelfish [21] and Popcorn Linux [20](Figure 2(b)), SSI OSes such as Locus [68], Hive [27], and fos [73] (Figure 2(d)) and disaggregated OSes such as LegoOS [60] (Figure 2(c)) use multiple kernel instances and coordinate them via message passing. In contrast, StarfishOS directly constructs kernel-space data structures in CXL for cross-machine coordination.

(2) A unique partial-failure model. Existing systems generally assume a single failure domain (for single-machine OSes, Figure 2(a) and (b)), or assume that when a machine fails, its CPU execution and OS state fail together (for SSIs and disaggregated OSes, Figure 2(d) and (c)). They all fail to handle the partial failure model introduced by CXL, where a machine's CPU and memory may fail while its kernel and application state remains accessible in CXL. In contrast, StarfishOS is designed explicitly for this partial-failure model.

(3) Supporting transparent cross-machine CPU resource disaggregation. Prior disaggregated systems mainly separate memory and storage while CPU execution remains organized per machine. For example, LegoOS [60] splits resources into CPU, memory, and SSD nodes, allowing multiple CPU nodes to share a common memory/storage pool, but each process still runs on a single CPU node. In contrast, StarfishOS extends disaggregation beyond memory and storage to CPUs while preserving transparency for applications with the conventional process/thread interface. Nu [57] also supports CPU resource disaggregation by decomposing applications into migratable logical processes, but requires applications to be reimplemented for its execution model.

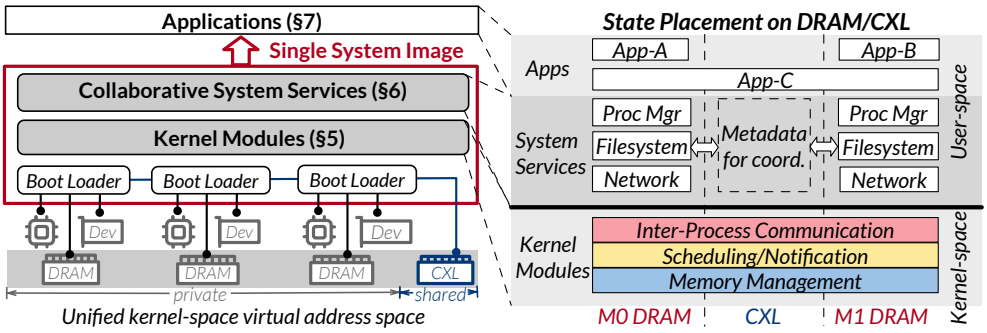


Figure 3. StarfishOS architecture. *StarfishOS is an OS-level SSI with unified kernel-space modules (§5), collaborative user-space system services (§6), and bootloaders (§4). With StarfishOS, applications (§7) run on a CXL pod as if they were running on a single machine.*

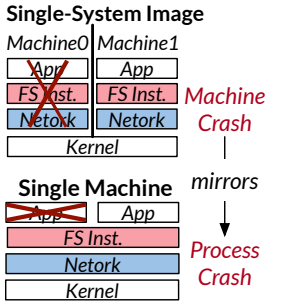


Figure 4. Failure model: *a machine crash in SSI behaves like the crash of processes in single-machine OS.*

Table 1. State partition and partial failure handling of each component in StarfishOS.

	Component	Runtime shared state	Partial failure handling
K-space func.	IPC (§5.1)	IPC queues & nodes	Detectably recoverable queues
	Notification (§5.3)	Waiting-lists & thread nodes	Detectably recoverable queues; libc timeouts mitigate higher-level primitives
	Scheduler (§5.2)	Inter-machine scheduling queues	Detectably recoverable queues
	Mem manager (§5.4)	CXL allocator's metadata/data	Lock-free buddy system & light undo logs
U-space	Sys services (§6)	Global coordination metadata	Micro-reboot service instances on surviving machines and repair state gaps.
	Applications (§7)	Process metadata (e.g., VMSpace) & part of heap	Applications recover with their own mechanisms; the OS remains usable on surviving machines and reclaims resources from failed ones.

4 Design Overview

4.1 StarfishOS in Action: A MapReduce Case

We use a simple MapReduce program to illustrate the core capabilities and components required by StarfishOS. Running MapReduce across a CXL pod requires:

- *Step 1: Create a thread pool and place mapper/reducer threads across machines.* This requires StarfishOS's ***scheduler*** to schedule threads on each CPU core in the pod.
- *Step 2: Initial threads load input data from the filesystem.* This requires (1) StarfishOS's ***filesystem services*** to provide a global filesystem namespace so that threads across the CXL pod can read the input file regardless of where it is physically stored, and (2) ***cross-machine IPC*** to invoke the filesystem service.
- *Step 3: Mappers compute data and place intermediate results into shared memory buffers.* This requires StarfishOS's ***memory manager*** to enable allocation of shared memory buffers from DRAM or CXL memory.
- *Step 4: After mappers complete, use condition variables to wake up reducers.* This requires a ***notification mechanism*** that supports waking up threads on other machines.

4.2 Basic Components of StarfishOS

As shown in Figure 3, StarfishOS consists of the following components to support the above workflow:

(1) *Bootloaders.* Each machine has a bootloader responsible for gathering local resources. Bootloaders coordinate with each other to boot StarfishOS, and they build a unified kernel virtual address space, so that machines across the pod execute in a consistent address layout.

(2) *Unified kernel-space modules (§5)* span multiple machines to provide core OS mechanisms, including IPC (*Step 2*), scheduling (*Step 1*), notification (*Step 4*), and memory management (*Step 3*). Together, these modules provide a unified kernel abstraction across the CXL pod, and their state is distributed across per-machine DRAM and CXL.

(3) *Collaborative system services (§6)* present a single global view across machines—a single PID space, filesystem namespace, and device namespace—so that applications can seamlessly use both local and remote services (for *Step 2*). We distribute collaborative service instances across machines, keeping instance state in DRAM and only coordination metadata in CXL.

(4) *Applications (§7)*, with the above three components, can transparently run across the CXL pod.

4.3 Hardware, Failure Model, Base Microkernel

Hardware setup. We target a CXL pod setup [74] (Figure 2 (e)) where rack-scale machines are equipped with *private memory* (i.e., local DRAM) and *shared memory* (i.e., memory connected through the CXL switch [3]).

We assume hardware cache coherence for CXL, but use only a small coherent region (several hundred MiBs). Although prior work [40] and vendors [41] suggest supporting coherence only for a small region, this mainly reflects hardware-complexity tradeoffs rather than a limitation: both

Arm's discussion [51] and FPGA prototypes [71] indicate that broader coherence is feasible. Still, StarfishOS minimizes coherent-memory usage to reduce coherence overhead [71].

We assume CPU caches are lost upon machine failures, and rely on cacheline flushes for partial-failure handling. Hardware support such as global persistent flush [58] could eliminate cacheline flushes and further improve performance.

Failure model. StarfishOS tolerates partial failures [80]. It assumes that CXL remains reliable and available through ECC and backup power supplies, as in prior work [77, 80], while physical machines may fail independently. It assumes fail-stop failures of entire physical machines (including CPU and DRAM), but not fail-slow or Byzantine failures [45]. It also assumes reliable machine-failure detection using heartbeats, as in prior work [84]. If CXL pool is nevertheless lost or corrupted, StarfishOS restarts the entire pod.

Behavior under partial failures. Under failures of one or several machines, StarfishOS preserves OS functionality on surviving machines, but does not provide transparent application recovery. (1) Preserving OS functionality means that all kernel-space mechanisms remain available on surviving machines, and system services become usable again after recovery. (2) No transparent application recovery means that an application continues unaffected only if none of its threads ran on the failed machine and none of its required state resided in that machine's DRAM. Otherwise, StarfishOS treats the whole affected application as failed, terminates its remaining threads, and reclaims its resources (Figure 4).

Background on the base microkernel. StarfishOS is built on ChCore [14, 35, 76], which is a capability-based microkernel like seL4 [42]. ChCore provides IPC, scheduler/notification, and memory management in kernel space, and system services such as process management and filesystems in user space. ChCore uses musl-libc [9] for POSIX compliance, and can transparently run various applications (e.g., Redis [10], Memcached [8], RocksDB [11]). For system services, StarfishOS provides user-space filesystem services (tmpfs/ext4) and the storage backend can be memory/SSD/JBOF [65]/RAID [54]. Its user-space network service is built on lwIP [32].

Although our prototype is built on a specific microkernel, the techniques described here also apply to others.

5 Kernel-Space Modules

This section describes how we realize SSI by extending kernel-space modules (IPC, scheduler, notification, and memory manager) to support cross-machine functionalities.

Our *core design principle* is to partition each kernel module's state into DRAM and CXL data structures for local and remote mechanisms (§ 5.1–§ 5.4), and make CXL data structures partial-failure resilient using lock-free, crash-consistent data structures [19, 33] (§5.5). Table 1 summarizes the methods used for each component.

5.1 Inter-Process Communication

IPC is the microkernel's mechanism for invoking system services. In our base kernel, IPC follows the L4/seL4 style [1, 22]: it is a function call-like, user-controlled context switch to a server thread without scheduler involvement. This design works for local IPC within a machine, but it does not extend naturally to cross-machine remote IPC when the server's address space and state remain in remote DRAM. We therefore keep the original design for local IPC, and implement cross-machine IPC as message passing over CXL.

Local IPC does not extend across machines (Figure 5 (a)). In our base kernel, local IPC follows the L4/seL4 style: the client transfers control directly to the server thread, which then executes in the server's address space. For a remote server, however, its address space and state remain on the remote machine, so switching control alone is not sufficient. A straightforward alternative is to place server state in CXL so that multiple machines can directly operate on it. We do not choose this design because: (1) it would consume a large amount of scarce cache-coherent CXL memory; (2) under partial failure, a machine crash may expose partially updated server state to other machines, making partial failures harder to recover from.

Remote IPC (Figure 5 (b)). We therefore adopt a CXL-based message-passing design for cross-machine IPC. As shown in Figure 5 (b), each service provides a dedicated handler thread for cross-machine IPC, separate from its local IPC. Each such handler thread consumes cross-machine IPC requests from a corresponding FIFO request queue in CXL. To invoke IPC, the client creates a message node on CXL (`struct Node`), prepares the request, enqueues it to the target machine's queue, and, if needed, waits for `status==DONE`. The server dequeues the node, handles the request, writes the response, and sets the status to `DONE` to notify the client.

5.2 Scheduling

StarfishOS extends thread scheduling beyond a single machine: it can assign a thread to any processing core in the CXL pod, including remote cores. Once assigned, the scheduler on the target core treats the thread as a local schedulable entity and schedules it according to the scheduling policy.

Local scheduling. As shown in Figure 6, the base microkernel uses per-core scheduling queues (*private queue*) that store runnable *thread control blocks*. A thread control block contains the execution state needed to resume a thread (e.g., saved registers), and the scheduling state (e.g., priority, affinity). Scheduler on each core selects a runnable thread from its private queue and context-switches to it.

Remote scheduling. To support scheduling a thread on remote machines, StarfishOS equips each core with an additional *shared queue* in CXL (Figure 6). Threads assigned to a remote core are enqueued into the shared queue, allowing the target core to admit them into its local queue for

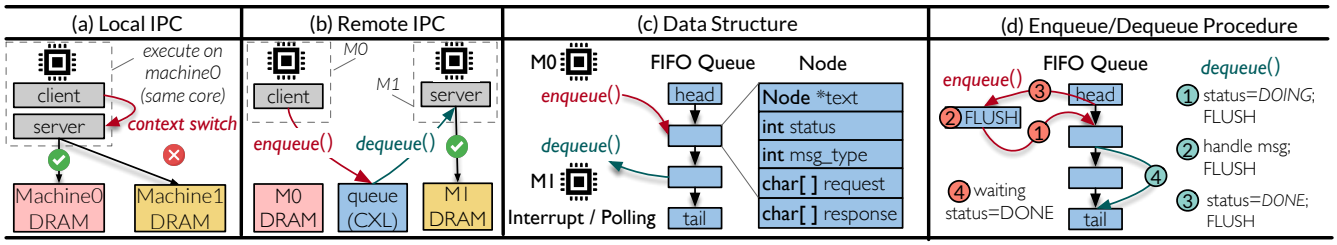


Figure 5. Inter-Process Communication.

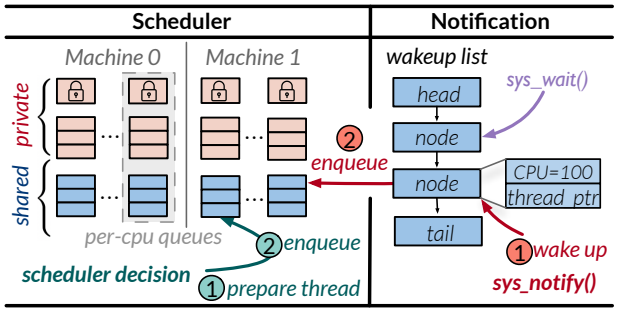


Figure 6. Scheduler/Notification.

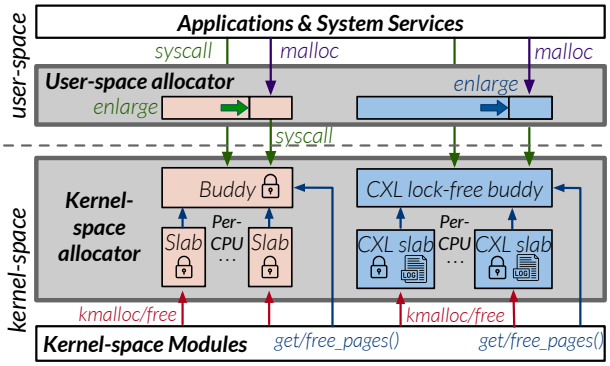


Figure 7. Memory Manager.

scheduling. The shared queue is checked along the target core's normal scheduling path, thus avoiding the overhead of sending explicit cross-machine scheduling requests.

By default, the scheduler is woken on each timer tick (e.g., 1 ms), so a newly enqueued remote thread could otherwise wait for one tick. To avoid this delay, StarfishOS sends a CXL MSI (delivered in 3-20 μs [39]) to interrupt the target CPU and promptly invoke its scheduler. This is analogous to mainstream kernels such as Linux, which use inter-processor interrupts (IPIs) to wake a scheduler on another CPU.

Note that a thread scheduled to a remote machine must already reside either in that machine's private memory (for notifications, §5.3) or in CXL (for cross-machine application execution, §7), so that the remote machine can access it.

Scheduling decisions are currently made globally in StarfishOS: when launching an application, the scheduler selects idle CPUs across the CXL pod for running it. Applications can also specify each thread's affinity (a global CPU ID), and the scheduler will migrate threads accordingly.

5.3 Notification

Notification is a mechanism for event-driven wakeup and serves as the basis for higher-level synchronization primitives (e.g., timeouts, futexes, and semaphores). StarfishOS extends notification to support cross-machine wakeup.

Local notification. As shown in Figure 6, our base microkernel maintains a *waiting-list* of threads waiting on a notification. To wait for an event, an application thread invokes `sys_wait` to insert its *thread control block* into the waiting-list. When the event occurs (i.e., `sys_notify` is invoked or a timeout fires), the kernel wakes one waiting thread by dequeuing it and scheduling it.

Remote notification. StarfishOS supports cross-machine notification by placing the notification's waiting-list in CXL. Thus, all machines can invoke `sys_wait`/`sys_notify` that operate on the shared waiting-list. However, the waking machine may still be unable to directly access the target thread's control block in another machine's private DRAM. To address this, each waiting-list node becomes a small *thread node*: it stores a pointer to the target thread together with which CPU the target thread is running on (`thread_ptr` and `CPU` in Figure 6), allowing the waking machine to enqueue the wakeup to the correct target CPU without directly dereferencing remote private thread state. Thus, a remote notifier never accesses a pointer stored in another machine's private DRAM; only the owner machine resolves it and accesses the local thread control block.

5.4 Memory Management

Memory management in StarfishOS supports both machine-local DRAM and pod-wide CXL memory.

Original memory management. As shown in the left part of Figure 7, similar to conventional kernels, the base microkernel uses a buddy/slab organization [23, 43] in kernel space and a libc allocator in user space. The kernel allocator serves kernel-space modules' allocations, while the user-space allocator obtains memory from the kernel and serves user-space processes' allocations.

CXL/DRAM memory management. StarfishOS extends memory management to manage both machine-local DRAM and pod-wide CXL. It uses a separate DRAM allocator on each machine and a shared CXL allocator for the entire pod.

In kernel space, StarfishOS uses conventional buddy/slab allocators for DRAM and replaces the buddy allocator with a lock-free allocator for CXL [75], which provides partial failure resilience (discussed later in §5.5). The slab allocator for CXL remains lock-based: each CPU has its own slab instance, so `malloc()` operations stay local and do not block allocation on other machines, preserving partial-failure resilience. When `free()` releases an object owned by another machine, it appends the deallocation request to that owner's log, and the owner machine reclaims the object asynchronously.

In user space, StarfishOS extends the libc allocator to manage both DRAM and CXL memory; applications can choose the target memory type when allocating memory, and the default is the DRAM of the machine on which the application runs. This memory-placement API is optional and unmodified applications can still be auto-scaled without using it.

5.5 Partial Failure Handling

Since StarfishOS places shared kernel state on CXL, it must ensure that partial failures neither corrupt shared state nor block surviving machines. For example, if a machine fails while holding a lock in CXL, the lock will not be released and may block other machines.

To avoid this, our *design principle* is to use lock-free and crash-consistent data structures for shared kernel state [19, 33]. Lock-freedom prevents failed machines from blocking others, while crash-consistency ensures that crashes do not leave shared state corrupted and that recovery can handle in-flight operations [19, 33].

Durable Queue for IPC, Scheduler, Notification (Appendix B). As listed in Table 1, we implement the IPC queue, scheduler queue, and notification waiting-list with DurableQueue, which is a lock-free and detectably recoverable FIFO queue proposed by Friedman et al. [33]. As shown in Figure 5 (d), enqueue and dequeue linearize with a single `CAS`, so a partial failure never leaves the queue itself corrupted or permanently blocked. Building on this property, StarfishOS handles failures differently depending on whether they occur at the enqueuer or dequeuer side:

- *Enqueuer Crash:* If the enqueuer's machine fails during `enqueue()`, the DurableQueue remains usable because `enqueue()` operation is lock-free and crash-consistent. Other machines can still enqueue to the DurableQueue.
- *Dequeuer Crash:* If the dequeuer's machine fails during `dequeue()`, other machines can continue operating, but the request may be lost or partially executed. Each component resolves this accordingly: IPC relies on service-specific recovery or client-side retry; the scheduler reconstructs its local state and retries; and notification returns `ABORT`, allowing libc to report a timeout.

Lock-free allocator (Appendix C). StarfishOS maintains a kernel-space buddy/slab allocator and a user-space libc allocator for CXL memory. We ensure their partial failure resilience as follows:

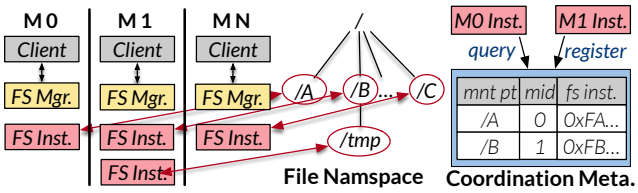


Figure 8. Collaborative filesystem service. *Filesystem instances are distributed across machines; each owns a disjoint part of the file namespace, as recorded in the coordination metadata.*

- *Lock-free Buddy:* We choose LLFree [75], a lock-free and crash-consistent allocator, to implement CXL buddy system. Under partial failure, the buddy state remains consistent, and other machines can still allocate/free memory without blocking since LLFree is lock-free.
- *Per-CPU Slab with Undo-log:* Each CPU has its own slab instance, so allocation on slab stays local and does not block allocation on other machines. Free operations on slab allocator are forwarded asynchronously to the owner machine. To safely reuse slab instances after recovery, StarfishOS keeps a lightweight in-memory undo-log to undo in-flight slab operations.
- *Libc allocator:* each application uses an independent libc allocator instance. Thus, the failure of libc allocator will not block other applications.

For memory leak prevention, allocator metadata remains consistent under partial failure, but memory may be allocated before it is linked to OS (e.g., a page allocated but not yet mapped into a process). StarfishOS prevents such leaks by using microkernel's capability-based state organization (§7.1) as the ground truth and checking allocator records against it during recovery.

6 Collaborative System Services

6.1 State Partition with Collaborative Instances

System services in StarfishOS are distributed as collaborative user-space instances. This keeps full service state in private DRAM and places only small coordination metadata in CXL. StarfishOS chooses to partition each system service into multiple owner instances instead of building one shared service for the whole pod because a single shared instance would push most service state into scarce cache-coherent CXL memory and make partial failures harder to contain: a machine crash during a service update could leave shared metadata in an inconsistent state that affects every client.

Each instance runs on one machine and owns a disjoint part of the global resource space, while the CXL metadata only records how to find that owner, e.g., mount-point-to-filesystem-instance mappings. Clients use this metadata to route service requests with local or cross-machine IPC, without sharing the instance's full internal state.

The filesystem in Figure 8 is the concrete example. Each machine runs a filesystem manager and several filesystem instances, and each instance owns one mount point. On the first access to a path, the client asks the manager which instance owns the mount point and establishes an IPC connection; later requests go directly to that instance. If the owner is remote, the request uses cross-machine IPC. Other system services follow the same pattern. Appendix D shows more details on filesystem service.

6.2 Partial Failure Handling

StarfishOS aims to preserve system-service availability for surviving applications under partial failures, including network communication, complete filesystem access, and process management. Partial failures are handled by recovering only the lost service instances. Because system services are user-space processes, StarfishOS can micro-reboot a failed instance, i.e., restart just that service instance on a surviving machine without restarting the whole pod [50].

Whether recovery is needed depends on the service type. Uniform services, such as networking, provide full functionality at every instance, so clients can reconnect to any surviving instance after a failure. Process management is handled similarly: clients can send requests to create a new process or reclaim application resources to a surviving manager. In contrast, non-uniform services whose instances own disjoint resources require explicit instance recovery. Recovering a stateful service instance requires mechanisms such as logging to reconstruct its state. In our current design, the filesystem is the only non-uniform service.

Uniform services. After a machine failure, clients can reconnect to the alive instance. The failure detector marks IPC connections to the failed instance as `ABORT`, and clients are notified and reconnect to a surviving instance.

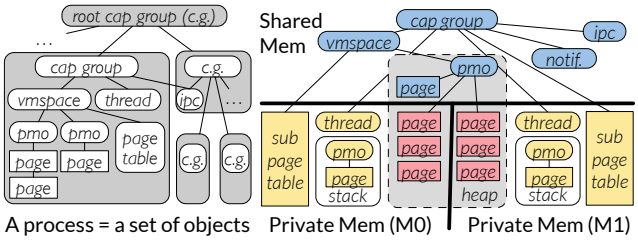
Non-uniform service (filesystem). Upon a machine failure, StarfishOS starts a replacement filesystem instance on another machine, and updates the mount-point mapping in the coordination metadata. Clients receive `ABORT` and reconnect to the replacement instance. To reconstruct its state, the instance first recovers from SSD and then replays its per-instance Ananke-style p-log [50] stored in CXL. The p-log records each mutating filesystem operation, including descriptor updates (e.g., `open`/`close`) and data or metadata updates (e.g., `write`/`mkdir`/`rename`). Thus, completed updates remain recoverable even if they have not reached SSD. The p-log's capacity is bounded at 4 MiB: when full, the filesystem discards entries superseded by later operations (e.g., state for a closed descriptor) and, if that is insufficient, syncs the remaining updates to SSD.

7 Cross-machine Applications

With the kernel modules and collaborative system services in place, applications can already run transparently on a CXL pod. The remaining question is where their state should live. StarfishOS therefore partitions application state for efficiency and defines how applications recover from partial failures.

Table 2. Objects managed by StarfishOS.

Object	Description
CapGroup	capability array that references other objects
Thread	execution context and scheduling state
VMSpace	virtual address-space metadata
PMO	physical memory pages
IPC	inter-process communication state
Notification	synchronization object



(a) Structured state **(b)** State placement of one process

Figure 9. Application state partition.

7.1 Application State Partition

State partitioning follows one principle: pay CXL latency only for state that must be shared across machines; keep all thread-local and machine-local state in DRAM.

Structured state organization. Capability-based microkernels [42, 61, 76, 79] represent system state as typed objects, and use capabilities to record which objects a process can reference and use. As shown in Figure 9(a), a process is therefore organized as a set of objects listed in Table 2. *CapGroup* stores the capabilities to these objects, making inter-object reachability explicit. This object-based organization lets StarfishOS track and place application state at object granularity.

Where to place each object? StarfishOS uses OS semantics to choose object placement. Figure 9(b) summarizes the resulting design along three cases:

(1) Private state stays in DRAM. Thread context and stack are per-thread and normally not shared between threads, so StarfishOS keeps them private to avoid CXL latency.[1] Besides, each machine has a private page table, which is described in next subsection (§7.2).

(2) Shared process metadata lives in CXL. Small and infrequently accessed process metadata (e.g., *CapGroup*, *VMSpace*, *PMO*) are placed in CXL. *IPC* and *Notification* state is also placed in CXL because these mechanisms are inherently cross-machine.

[1] In some legacy POSIX applications, one thread may access another thread's stack. StarfishOS still handles this case: the access triggers a page fault that migrates the referenced stack page to CXL.

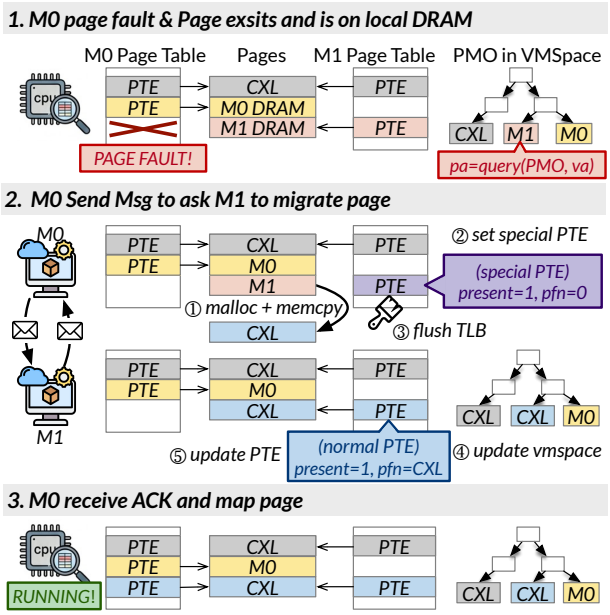


Figure 10. Transparent page placement across DRAM and CXL with Page-fault-driven migration.

(3) Pages of mixed regions are shared on demand. The heap, code segment, and variable regions (.bss/.data) mix local data with data requiring cross-machine access, and placing the whole region in CXL would sacrifice application performance. StarfishOS therefore uses page-fault-driven migration to keep heap/code-segment/variables pages in DRAM by default and move only remotely accessed pages to CXL.

7.2 On-demand Sharing of Heap/Code/Variables

StarfishOS uses page-fault-driven migration to transparently share heap pages, code segments, and variables (.bss/.data) on demand. By default, a page is allocated in private DRAM of the machine that first touches it. When a thread accesses a page in another machine's DRAM, StarfishOS migrates the page to CXL upon a page fault. To enable this mechanism, StarfishOS maintains a local page-table replica (i.e., a "sub page table", M0/M1 page table in Figure 10) on each machine running the application. All sub page tables map the same global virtual address space, but each entry may temporarily point to a local DRAM page or its CXL replacement.

DRAM-to-CXL migration. Figure 10 shows the detailed migration procedure when machine0 accesses a page currently owned by machine1:

1. Machine0 faults on a virtual address and queries *VMSpace* to determine whether the address has a valid mapping and, if so, whether the mapping points to a DRAM page. If so, machine0 identifies the owner machine and sends it a migration request through the IPC mechanism in §5.1.
2. Machine1 handles the migration request. It allocates a CXL page and copies the page data from DRAM to CXL (①). It then atomically replaces the original PTE (page table entry) with a migration entry (②), a special illegal PTE [15, 16] that blocks concurrent accesses until the migration finishes. After flushing stale TLB entries (③), machine1 updates *VMSpace* (④) and installs a new PTE that points to the CXL page (⑤). Any access that encounters the migration entry waits for completion, then observes the new mapping and refills TLB.
3. After receiving the ACK together with the CXL page's physical address, machine0 installs its local mapping to the CXL page and resumes execution.

StarfishOS supports concurrent page faults from different machines on the same page by maintaining a per-`VMSpace` list of in-flight migrations; a fault that finds an existing migration waits for it to finish.

CXL-to-DRAM migration. When CXL memory is nearly full (e.g., 80%), StarfishOS frees space by migrating CXL-resident pages back to private DRAM and invalidating stale mappings, as in DRAM-to-CXL migration. It selects pages to migrate back in the order in which they entered CXL, and prefers pages shared by fewer machines. To amortize page-fault handling and TLB-flush overheads, StarfishOS performs CXL-to-DRAM migration in batches.

StarfishOS does not choose eager migration. Although eager migration may reduce subsequent access latency, migrating a CXL-resident page—which may still be accessed by other machines—back to DRAM can incur unnecessary copying and page-table updates, potentially causing repeated migrations and degrading application performance.

7.3 Handling Partial Failure of Applications

StarfishOS does not hide partial failures from applications. An application whose threads and required state remain on surviving machines can continue running; otherwise, StarfishOS treats the entire cross-machine application as failed and terminates its remaining threads. Consequently, application-private state, such as POSIX mutexes, need not remain usable under partial failure. StarfishOS does not provide transparent recovery of applications because transparent recovery would add common-case cost, and many applications already use write-ahead logging, restart, or replay from saved intermediate state.

For cross-machine applications, StarfishOS automatically reclaims resources after a machine failure: StarfishOS records each application's *CapGroup* in CXL, allowing surviving machines to traverse the state reachable from the *CapGroup* and reclaim thread contexts, pages, and IPC/notification state.

8 Putting It All Together

How StarfishOS partitions state. StarfishOS applies one *placement principle* across all layers: state accessed only by one machine is placed in DRAM, while state for cross-machine access/coordination or recovery is placed in CXL. Figure 11 summarizes the resulting placement across layers.

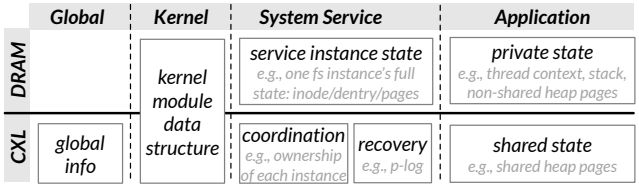


Figure 11. State partition overview.

OS procedure after partial failures. Upon detecting a machine failure, StarfishOS marks its service instances offline in *coordination metadata* (see Figure 8) and changes remaining requests in IPC connections to these instances from `DOING` to `ABORT`. StarfishOS then restarts the affected service instances on surviving machines and updates instance information in *coordination metadata* for client reconnection.

For cross-machine applications, because their execution state has been lost, StarfishOS terminates their surviving threads and reclaims their resources through capability-based recycling. The reduced pod can then continue running new or restarted applications.

Once the failed machine is repaired, StarfishOS boots it with the same logical machine ID and an incremented generation, distinguishing the new incarnation from stale state. The rejoining kernel rebuilds its local volatile state, including IPC, scheduler, and allocator state, while attaching to the existing CXL-resident state. It then rejoins the pod, and its CPUs and local memory become available to the pod again.

9 Evaluation

We evaluate StarfishOS using microbenchmarks and real applications to answer the following questions:

- How do the basic modules of StarfishOS perform? (§9.2)
- How does state partitioning contribute to efficiency? (§9.3)
- How can applications benefit from StarfishOS in auto-scaling and resource utilization use cases? (§9.4 and §9.5)
- How do applications recover from failures? (§9.6)

9.1 Experimental Setup

Testbed. Our evaluation platform has four Intel® Xeon® Gold 6418H CPUs (Sapphire Rapids), each with 24 physical cores and 64 GiB of DRAM. The server also includes a real 128 GiB CXL 1.1 memory device. CPU frequency is fixed at 2.1 GHz with Hyper-threading disabled, Intel® Turbo Boost disabled and NUMA balancing disabled.

Environment. Due to the lack of real CXL switch hardware, we emulate an 8-machine CXL pod by running 8 QEMU/KVM instances, each with 12 physical CPU cores. Every two emulated machines share a NUMA node, and all machines access a CXL memory device. Except for applications running on StarfishOS, all other experiments are conducted on Linux v5.15 with Ubuntu 22.04. Among them, Tigon [40] is evaluated in Linux VMs to emulate a CXL environment, while the other baselines run directly on bare metal without virtualization. Each experiment is performed 3 times following an initial warm-up run. VM exits account for 3–5% of execution time across the applications we evaluate.

Table 3. DRAM/CXL latency/bandwidth of our CXL 1.1 device and reported CXL 3.0 FPGA/ASIC results. *Local and remote stand for local and remote NUMA. FPGA performance is based on a real FPGA-based CXL 3.0 prototype [71], and ASIC performance is based on reports from IP providers [31]. Bandwidth: GiB/s/channel for DRAM, GiB/s/line for CXL.*

Metric	Local DRAM	Remote DRAM	Local CXL	Remote CXL	FPGA CXL3.0	ASIC CXL3.0
Latency (ns)	107	160	241	383	456	170
Bandwidth (GiB/s)	52	30	2	0.3	32	32

CXL memory performance. As shown in Table 3, compared to CXL 3.0, our CXL 1.1 environment has comparable latency (241/384 ns v.s. 456 ns in FPGA and 170 ns in ASIC), while the bandwidth is substantially lower (2/0.3 GiB/s v.s. 32 GiB/s in FPGA and ASIC). Besides, our platform does not model the cacheline invalidation overhead in CXL 3.0 [71], which increases cache miss latency by 2–3× [71]. Nevertheless, these differences do not affect the conclusions of our evaluation: the lower bandwidth only puts our system at a disadvantage, and the latency is comparable; moreover, our partitioning strategy keeps most accesses on local DRAM and limits CXL accesses to a small fraction of the workload, thereby reducing the impact of the cacheline invalidation overhead in CXL 3.0 (for example, Matrix Multi. has a cache-miss rate below 5% when accessing CXL in our tests, resulting in an end-to-end runtime increase of at most 10%).

Workloads. We evaluate StarfishOS using microbenchmarks and the following end-to-end workloads. All applications running on StarfishOS have no code changes.

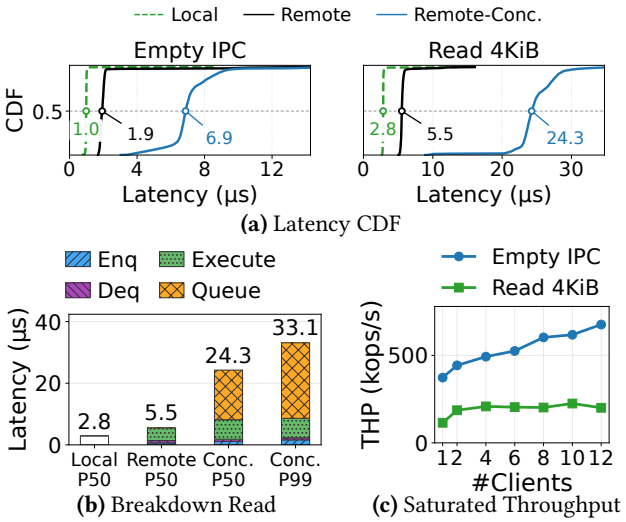
- *Phoenix-2.0 [55]*: a single-machine MapReduce application.
- *DB×1000 [6, 78]*: a single-machine OLTP database system designed to make DBMS scalable on 1000-core processors.
- *GeminiGraph [83]*: a graph processing application with both single-machine and distributed modes.
- *CNN [2]*: a single-machine neural-network inference.
- *Redis [10]*, *Memcached [8]* and *LevelDB [7]*: memory-based and disk-based key-value stores.

9.2 Microbenchmarks

This section evaluates the basic kernel modules.

IPC. We measure local and remote IPC with an *empty* call and a *read 4 KiB* filesystem call; remote IPC uses either one sender (*Remote*) or four concurrent senders (*Remote-Conc.*). For the empty call, P50 latency is 1.0 μs locally and 1.9/6.9 μs remotely without/with concurrency; for *read 4 KiB*, it is 2.8 and 5.5/24 μs, respectively (Figure 12(a)).

Breakdown (Figure 12(b)) shows that the additional tail latency comes primarily from service-side serialization under concurrency (labeled as *Queue*, 16 μs at Conc.-P50 and 24 μs at Conc.-P99), not from the extra cacheline flushes required by the recoverable queue since enqueue/dequeue costs 1.4 μs.



(a) Latency CDF

(b) Breakdown Read

(c) Saturated Throughput

Figure 12. IPC latency and throughput.

Table 4. Scheduling/Notification P50 latency (μs).

	Local (use 5 μs IPI)	Remote (use 10 μs MSI)
Scheduling	19.4	65.2
Notification	15.3	22.8

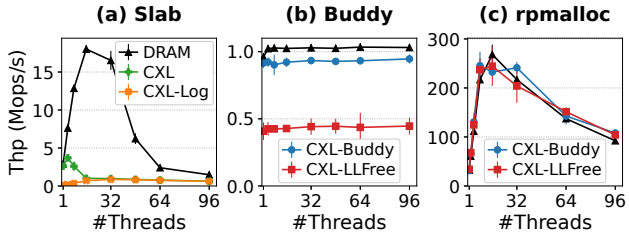


Figure 13. Memory allocator throughput.

Figure 12(c) shows server-side throughput of empty/read-4 KiB IPC as the number of concurrent clients increases. The throughput eventually saturates as clients increase; further clients primarily increase queueing latency.

Scheduling/Notification. Table 4 shows end-to-end latency from a thread calling *sys_set_affinity* until it runs on the target core, and from a notifier calling *sys_notify* until the waiting thread is woken and runs. The local/remote gap (19.4 μs vs. 65.2 μs for scheduling, 15.3 μs vs. 22.8 μs for notification) comes mainly from the interrupt that wakes the target scheduler: local operations use an IPI (about 5 μs), whereas remote operations use a CXL MSI (about 10 μs). Remote scheduling may wait for runnable local threads to be serviced, which adds additional latency.

Memory Allocation. We evaluate *Slab*, *Buddy*, and our libc allocator *rpmalloc*. *Slab* and *rpmalloc* allocate 64–1024 B objects following the rpmalloc benchmark [12], while *Buddy* allocates 4 KiB/2 MiB pages with equal probability. For each allocator, we compare the base kernel's *DRAM* allocator with CXL configurations backed by either a conventional buddy allocator (*CXL-Buddy*) or our LLFree-based buddy allocator (*CXL-LLFree*) [75]. For Slab with CXL-LLFree, we further compare CXL without undo logging (*CXL* in (a)) and with undo logging (*CXL-Log* in (a)).

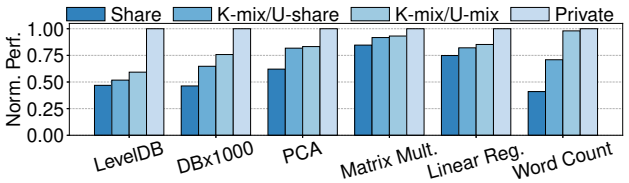


Figure 14. Performance across state-partition choices.

Figure 13 shows that kernel-space allocation is more sensitive to CXL placement than user-space allocation. The impact on user-space *rpmalloc* is smaller because *rpmalloc* mostly serves `malloc`/`free` from thread-local free lists, and therefore touches cached local metadata more often than the kernel allocator. Besides, *Undo-log* and *CXL-LLFree* add cache-line flushes, reducing kernel-space allocator's performance, but have minimal impact on user-space *rpmalloc*.

9.3 Effect of State Partitioning

We evaluate the effectiveness of StarfishOS's state partitioning by running several applications across 8 machines with four state-partition configurations: (1) *Share*: place all kernel and application state in CXL; (2) *K-mix/U-share*: partition only kernel state, while placing all application state in CXL; (3) *K-mix/U-mix*: partition both application and kernel state; (4) *Private*: all state in private DRAM (ideal case).

Figure 14 reports normalized performance for four state-partition choices, using *Private* as the baseline (throughput for LevelDB and DB×1000; 1/*execution_time* for the other applications). *K-mix/U-mix* partitioning outperforms *Share* by up to 139%, and outperforms *K-mix/U-share* by up to 38%, indicating the effectiveness of state partitioning.

9.4 Auto-scaling Applications

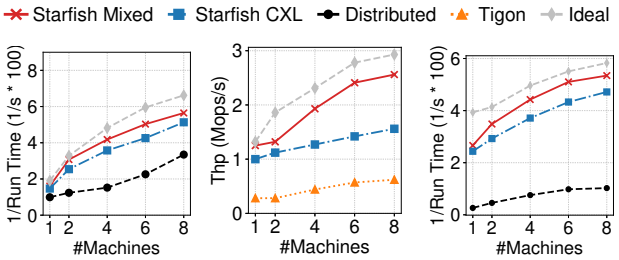
This case study evaluates whether unmodified single-machine apps can auto-scale on StarfishOS and beat distributed and CXL-optimized baselines.

We select applications that already scale well to reveal whether StarfishOS can preserve application scalability when running them transparently across machines. In contrast, StarfishOS cannot improve the scalability of applications with an inherent scalability bottleneck.

Configs. We use 3 applications: (1) Phoenix [55]'s Matrix Multi. (multiply one 4000×4000 matrix); (2) DB×1000 [6, 78] running TPCC [13] with 64 warehouses, 50% New-order, 50% Payment, 15% cross-warehouse transactions; and (3) Gemini-Graph [83] running PageRank [53] on twitter-2010 [44].

We test two StarfishOS configs: *StarfishOS-CXL*, which places all heap and page-table state in CXL, and *StarfishOS-Mixed*, which places only shared part of the heap in CXL.

We compare StarfishOS with distributed baselines (*Distributed*) for all applications, and compare DB×1000 running on StarfishOS with Tigon [40]. For Phoenix, we compare



(a) Matrix Mult. **(b)** DB×1000 **(c)** GeminiGraph

Figure 15. Auto-scaling single-machine applications.

Table 5. Mem usage in auto-scaling apps (8 machines).
We bound DB×1000 and Tigon [40]'s CXL usage to 500 MiB.

Mem	Matrix Mult.	DB×1000	Tigon	GeminiGraph
Total	*1 GiB*	*26 GiB*	*26 GiB*	*9 GiB*
CXL	*109 MiB*	*500 MiB*	*500 MiB*	*600 MiB*

with a multi-process local TCP-based baseline. For DB×1000, we compare with Tigon [40]'s open-source implementation using the same settings (TPCC workload with 64-warehouses and 15% cross-warehouse transactions, WAL-disabled, and running in VMs). For GeminiGraph, which provides both single-machine and distributed modes, we compare with its native distributed MPI implementation.

All measured configurations use multiple VMs because the requested CPU parallelism does not fit in one VM. *Ideal* is not an executable configuration, but a hypothetical Linux setup with enough CPUs and all state in private DRAM, serving as an upper-bound reference.

Scaling. Figure 15 shows that applications can scale from 1 to 8 machines with 3.5× (Matrix Mult.), 2.1× (DB×1000), and 2.0× (GeminiGraph) speedup under *StarfishOS-Mixed*. Across all three applications, *StarfishOS-Mixed* consistently outperforms *StarfishOS-CXL*, indicating the effectiveness of state partitioning. Besides, StarfishOS-Mixed achieves 85%–92% of the ideal performance on 8 machines while consuming only several hundred MiB of CXL memory, compared with total working set size of 1–26 GiB, as shown in Table 5.

StarfishOS v.s. distributed baselines. Applications running on StarfishOS achieve 5.2× (GeminiGraph) and 1.7× (Matrix Mult.) speedup over their distributed baselines. The gain comes from replacing network communication with direct CXL load/store: Phoenix avoids MapReduce shuffle copies, DB×1000 avoids network-based 2PC via shared state, and GeminiGraph lets partitions update remote vertices without message passing.

StarfishOS v.s. CXL-optimized baseline. StarfishOS achieves 4.1× the throughput of Tigon at 8 machines and 87% of Ideal, showing that transparent placement already captures much of the benefit of CXL-based shared-state access. This result is notable because Tigon is an application-specific redesign using application semantics to place cross-machine shared data in CXL on demand. Note that this is not a strictly

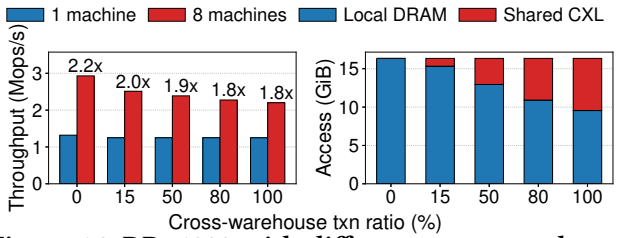


Figure 16. DB×1000 with different cross-warehouse transaction ratios.

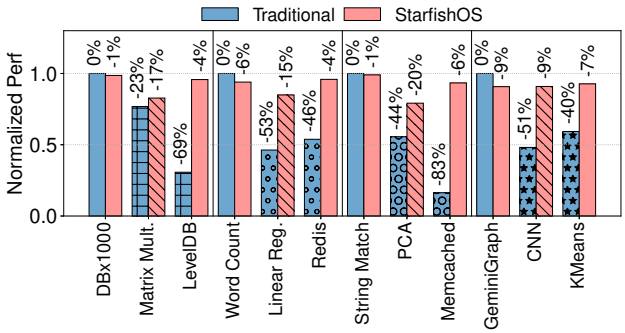


Figure 17. CPU Resource Utilization.

apples-to-apples comparison, since Tigon and DB×1000 are based on different codebases. In our test, Tigon is bottlenecked by limited intra-machine scalability.

TPCC with different cross-warehouse transaction ratios. Figure 16 further shows that StarfishOS remains effective as more data must be accessed from CXL. We vary the cross-warehouse transaction ratio from 0% to 100% on 8 machines with all other settings the same as in Figure 15, reporting throughput and the volume of accesses to objects that reside in DRAM or CXL (not the CXL capacity, which is fixed to 1 GiB). StarfishOS achieves 2.0× speedup at the default 15% ratio, 1.8× at 80%, and 1.76× at 100%, while CXL access grows from about 1.0 GiB at 15% to 5.4 GiB at 80% as more cross-warehouse shared objects are placed in CXL.

9.5 CPU Resource Utilization

StarfishOS enables applications to utilize stranded CPU resources by running applications across multiple machines. In Figure 17, we evaluate 12 datacenter applications on traditional OSes (our base microkernel) and StarfishOS. Each application has 8 threads, and we have 8 machines, each with 12 cores. Under traditional OSes, each application is restricted to a single machine. Taking three applications and two machines as a group, one application runs exclusively on one machine (non-hatched bars in *Traditional*), while the other two applications are co-located on the other machine (hatched bars in *Traditional*). Bars with the same hatch pattern indicate applications co-located on the same machine. In contrast, with StarfishOS, applications can span multiple machines without requiring any code modifications. Consequently, in each group, two applications run independently on separate machines (unhatched bars in *StarfishOS*),

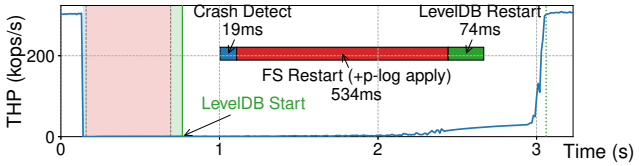


Figure 18. Recovery of LevelDB. *The blue line reports LevelDB throughput over time; the shaded boxes mark failure detection, filesystem micro-reboot (p-log replay) and LevelDB recovery.*

while the remaining application executes across both machines (hatched bars in *StarfishOS*). All results in Figure 17 are normalized to the performance of exclusive execution of 8-threaded applications on 8 cores on our base microkernel.

The results demonstrate that StarfishOS can effectively utilize stranded CPU resources scattered across multiple machines. As shown in Figure 17, co-located applications on a traditional OS (traditional-hatched) achieve only 17%–77% of the performance of exclusive execution. In contrast, applications that run across machines on StarfishOS (StarfishOS-hatched) achieve 83%–93% of exclusive performance, while other eight applications (StarfishOS-unhatched) are nearly unaffected. These results highlight the benefit of StarfishOS's ability to utilize all stranded CPU resources with acceptable performance downgrades. We further evaluated several alternative grouping and co-location strategies, all of which exhibit similar performance gains with StarfishOS, reinforcing the generality of these benefits.

9.6 Partial Failure Resilience

To evaluate how applications recover from partial failures of machines, we deploy LevelDB on one machine and crash that machine by killing that QEMU instance. After 19 ms, our failure detector detects the machine failure and micro-reboots a filesystem instance on another machine in 5.5 ms. The replacement filesystem instance first reconstructs its state from the state flushed to disk and then uses Ananke-style replay [50] to recover the gap between the flushed state and the crashed instance's latest state. Specifically, it replays the p-log in 528 ms (to replay 446 entries after LevelDB's last flush). LevelDB can then reconstruct its in-memory state in 73 ms and warm up its execution in 3 s.

10 Discussion and Related Work

Adapting StarfishOS to CXL without cache coherence. On CXL without even a small cache coherence region, StarfishOS can still use shared CXL queues for cross-machine kernel module functions and system services by adapting data structures to non-cache-coherent hardware. However, unmodified shared-memory processes cannot run transparently across machines without additional support.

CXL's NUMA effects. CXL exhibits non-uniform access latency and bandwidth due to topology asymmetry. These effects are orthogonal to StarfishOS's core contribution: StarfishOS can apply existing NUMA-aware techniques to keep computation near private state and place shared pages on nearby CXL devices.

Future directions. StarfishOS currently provides transparent cross-machine execution and partial failure resilience of kernel functions and system services. It does not make application state or POSIX locks resilient, and there is still room for improving scheduling policy, migration efficiency, and system service scalability. We will continue to improve efficiency and reliability of StarfishOS in future work.

Other related works. StarfishOS builds on several lines of prior work, but combines them in a different setting. Our cross-machine kernel modules and collaborative services build on decades of SSI design [20, 27, 49, 68, 73]. Our state partitioning draws inspiration from SASOS [29, 37, 47], Corey [24], and Mitosis [17], which highlight the benefits of explicit state management. For partial-failure resilience, StarfishOS adopts lock-free, detectably recoverable data structures [19, 33] and lock-free allocators [75] motivated by recent work on CXL partial-failure resilience [72, 80, 84]. Our page-fault-triggered migration and replicated page-table design are also inspired by prior NUMA and page-table replication work [17]. Our contribution is integrating these ideas into an OS-level SSI that transparently runs unmodified applications on a CXL pod while preserving availability under partial failures. For resource disaggregation, SigmaOS [67] uses cloud procs, Nu [57]/Quicksand [56] use logical processes or components, and Arca [64] uses portable continuations. These models enable fine-grained resource management but require rewriting applications to adopt their respective execution models; StarfishOS instead transparently runs existing applications.

11 Conclusion

StarfishOS is the first OS-level SSI to use CXL's shared-memory primitive for efficient cross-machine synchronization. Built on a state-partitioned microkernel, it keeps CXL-resident state small, provides OS-level partial-failure resilience, and transparently scales single-machine applications across multiple machines to achieve substantial speedups and utilize pod-wide device resources.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback and thank our shepherd Simon Peter for the detailed and insightful suggestions that significantly improved the paper. We also thank Jiaohao Zhou, Dong Du, and Hongtao Lyu for their valuable suggestions. This work was supported in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM113) and the National Natural Science Foundation of China (No. 62132014 and 624B2095). Corresponding authors: Mingkai Dong (mingkaidong@sjtu.edu.cn) and Haibo Chen (haibochen@sjtu.edu.cn).

References

[1] 2019. How to and how not to use seL4 IPC. https://microkerneldude.org/2019/03/07/how-to-and-how-not-to-use-sel4-ipc/.

[2] 2025. CNN. https://github.com/ekut-es/pico-cnn.

[3] 2025. Compute Express Link (CXL). https://www.computeexpresslink.org/.

[4] 2025. CXL Consortium. Compute Express Link Specification, Revision 3.1, 2025. https://computeexpresslink.org/wp-content/uploads/2025/02/CXL-3.1-Specification.pdf.

[5] 2025. *CXL Use Case: Ray*.

[6] 2025. DBx1000. https://github.com/yxymit/DBx1000.

[7] 2025. LevelDB. https://dbdb.io/db/leveldb.

[8] 2025. Memcached - a distributed memory object caching system. https://memcached.org/.

[9] 2025. Musl Libc. https://musl.libc.org/.

[10] 2025. Redis. https://redis.io/.

[11] 2025. RocksDB Home Page. https://www.rocksdb.org.

[12] 2025. Rpmalloc Benchmark. https://github.com/mjansson/rpmalloc-benchmark.

[13] 2025. TPC Benchmark C. https://www.tpc.org/tpcc/.

[14] 2025. TreesLS. https://ipads.se.sjtu.edu.cn/projects/treesls.html.

[15] 2026. Page migration. https://docs.kernel.org/mm/page_migration.html.

[16] 2026. Process Addresses. https://docs.kernel.org/mm/process_addrs.html.

[17] Reto Achermann, Ashish Panwar, Abhishek Bhattacharjee, Timothy Roscoe, and Jayneel Gandhi. 2020. Mitosis: Transparently Self-Replicating Page-Tables for Large-Memory Machines. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 283–300. doi:10.1145/3373376.3378468

[18] C. Amza, A.L. Cox, S. Dwarkadas, P. Keleher, Honghui Lu, R. Rajamony, Weimin Yu, and W. Zwaenepoel. 1996. TreadMarks: Shared Memory Computing on Networks of Workstations. 29, 2 (1996), 18–28. doi:10.1109/2.485843

[19] Hagit Attiya, Ohad Ben-Baruch, Panagiota Fatourou, Danny Hendler, and Eleftherios Kosmas. 2022. Detectable recovery of lock-free data structures. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) *(PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 262–277. doi:10.1145/3503221.3508444

[20] Antonio Barbalace, Marina Sadini, Saif Ansary, Christopher Jelesnianski, Akshay Ravichandran, Cagil Kendir, Alastair Murray, and Binoy Ravindran. 2015. Popcorn: bridging the programmability gap in heterogeneous-ISA platforms. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) *(EuroSys '15)*. Association for Computing Machinery, New York, NY, USA, Article 29, 16 pages. doi:10.1145/2741948.2741962

[21] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhania. 2009. The multikernel: a new OS architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) *(SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 29–44. doi:10.1145/1629575.1629579

[22] B. Bershad, T. Anderson, E. Lazowska, and H. Levy. 1989. Lightweight remote procedure call. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP '89)*. Association for Computing Machinery, New York, NY, USA, 102–113. doi:10.1145/74850.74861

[23] Jeff Bonwick. 1994. The slab allocator: an object-caching kernel memory allocator *(USTC'94)*. USENIX Association, USA, 6.

[24] Silas Boyd-Wickizer, Haibo Chen, Rong Chen, Yandong Mao, Frans Kaashoek, Robert Morris, Aleksey Pesterev, Lex Stein, Ming Wu, Yuehua Dai, Yang Zhang, and Zheng Zhang. 2008. Corey: an operating system for many cores. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) *(OSDI '08)*. USENIX Association, USA, 43–57.

[25] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. 2018. Efficient Distributed Memory Management with RDMA and Caching. 11, 11 (2018), 1604–1617. doi:10.14778/3236187.3236209

[26] George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox. 2004. Microreboot—A Technique for Cheap Recovery. In *6th Symposium on Operating Systems Design and Implementation (OSDI '04)*. USENIX Association, San Francisco, CA.

[27] J. Chapin, M. Rosenblum, S. Devine, T. Lahiri, D. Teodosiu, and A. Gupta. 1995. Hive: fault containment for shared-memory multiprocessors. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (Copper Mountain, Colorado, USA) *(SOSP '95)*. Association for Computing Machinery, New York, NY, USA, 12–25. doi:10.1145/224056.224059

[28] Matthew Chapman and Gernot Heiser. 2005. Implementing Transparent Shared Memory on Clusters Using Virtual Machines. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. USENIX Association, Anaheim, CA. https://www.usenix.org/conference/2005-usenix-annual-technical-conference/implementing-transparent-shared-memory-clusters

[29] Jeffrey S. Chase, Henry M. Levy, Michael J. Feeley, and Edward D. Lazowska. 1994. Sharing and Protection in a Single-Address-Space Operating System. *ACM Trans. Comput. Syst.* 12, 4 (November 1994), 271–307.

[30] David Cheriton. 1988. The V distributed system. *Commun. ACM* 31, 3 (mar 1988), 314–333. doi:10.1145/42392.42400

[31] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (July 2024), 37 pages. doi:10.1145/3669900

[32] Adam Dunkels. 2001. lwIP: A Lightweight TCP/IP Stack. https://www.nongnu.org/lwip/ Accessed: 2026-07-24.

[33] Michal Friedman, Maurice Herlihy, Virendra Marathe, and Erez Petrank. 2018. A persistent lock-free queue for non-volatile memory. (2018), 28–40. doi:10.1145/3178487.3178490

[34] Jim Gray. 1978. *Notes on Data Base Operating Systems.* Technical Report. Berlin, Heidelberg. 393–481 pages.

[35] Jinyu Gu, Xinyue Wu, Wentai Li, Nian Liu, Zeyu Mi, Yubin Xia, and Haibo Chen. 2020. *Harmonizing Performance and Isolation in Microkernels with Efficient Intra-Kernel Isolation and Communication.* USENIX Association.

[36] Gernot Heiser and Kevin Elphinstone. 2016. L4 Microkernels: The Lessons from 20 Years of Research and Deployment. *ACM Trans. Comput. Syst.* 34, 1, Article 1 (apr 2016), 29 pages. doi:10.1145/2893177

[37] Gernot Heiser, Kevin Elphinstone, Stephen Russell, and Jerry Vochteloo. 1994. Mungi: A Distributed Single-Address-Space Operating System. In *Proceedings of the 17th Australasian Computer Science Conference (ACSC)*. Christchurch, New Zealand, 271–80.

[38] Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg, and Andrew S. Tanenbaum. 2007. Failure Resilience for Device Drivers. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*. IEEE, 41–50.

[39] Craig Howard. 2011. What Is a Ballpark Figure for PCIe Interrupt-to-Userspace Latency? https://community.osr.com/t/whats-a-ballpark-figure-for-pcie-interrupt-touserspace-latency/41717. Accessed: 2024-07.

[40] Yibo Huang, Haowei Chen, Newton Ni, Yan Sun, Vijay Chidambaram, Dixin Tang, and Emmett Witchel. 2025. Tigon: a distributed database

for a CXL pod. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) *(OSDI '25)*. USENIX Association, USA, Article 7, 20 pages.

[41] Sunita Jain, Nagaradhesh Yeleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory Sharing with CXL: Hardware and Software Design Approaches. arXiv:2404.03245 [cs.ET] https://arxiv.org/abs/2404.03245

[42] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. SeL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) *(SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 207–220.

[43] Kenneth C. Knowlton. 1965. A fast storage allocator. *Commun. ACM* 8, 10 (oct 1965), 623–624. doi:10.1145/365628.365655

[44] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media?. In *Proceedings of the 19th International Conference on World Wide Web* (Raleigh, North Carolina, USA) *(WWW '10)*. Association for Computing Machinery, New York, NY, USA, 591–600. doi:10.1145/1772690.1772751

[45] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401. doi:10.1145/357172.357176

[46] Seung-seob Lee, Yanpeng Yu, Yupeng Tang, Anurag Khandelwal, Lin Zhong, and Abhishek Bhattacharjee. 2021. MIND: In-Network Memory Management for Disaggregated Data Centers. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (New York, NY, USA, 2021-10-26). Association for Computing Machinery, 488–504. doi:10.1145/3477132.3483561

[47] Ian Leslie, Derek McAuley, Richard Black, Timothy Roscoe, Paul Barham, David Evers, Robin Fairbairns, and Eoin Hyden. 1996. The Design and Implementation of an Operating System to Support Distributed Multimedia Applications. *IEEE Journal on Selected Areas in Communications* 14 (1996), 1280–1297.

[48] Kai Li. 1988. IVY: A Shared Virtual Memory System for Parallel Computing. In *International Conference on Parallel Processing*. https://api.semanticscholar.org/CorpusID:29219727

[49] Felix Xiaozhu Lin, Zhen Wang, and Lin Zhong. 2014. K2: a mobile operating system for heterogeneous coherence domains. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) *(ASPLOS '14)*. Association for Computing Machinery, New York, NY, USA, 285–300. doi:10.1145/2541940.2541975

[50] Jing Liu, Yifan Dai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2025. Fast, Transparent Filesystem Microkernel Recovery with Ananke. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 1–18. https://www.usenix.org/conference/fast25/presentation/liu-jing

[51] Arm Ltd. 2024. Arm OCP CMS – Design Considerations in Adding Coherency to CXL Devices (HDM-DB). OCP-CMS Workgroup Presentation. https://ocp-all.groups.io/g/OCP-CMS/attachment/222/1/Arm%20OCP%20CMS%20-%20Design%20Considerations%20in%20Adding%20Coherency%20to%20CXL%20Devices%20(HDM-DB)%20-%20Jan%2012%202024.pdf OCP-CMS meeting, Jan. 12, 2024.

[52] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. 2015. Latency-Tolerant Software Distributed Shared Memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. USENIX Association, Santa Clara, CA, 291–305. https://www.usenix.org/conference/atc15/technical-session/presentation/nelson

[53] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking : Bringing Order to the Web. In *The Web Conference*. https://api.semanticscholar.org/CorpusID:1508503

[54] David A. Patterson, Garth A. Gibson, and Randy H. Katz. 1988. A Case for Redundant Arrays of Inexpensive Disks (RAID). In *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. ACM, 109–116. doi:10.1145/50202.50214

[55] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradski, and Christos Kozyrakis. 2007. Evaluating MapReduce for Multi-core and Multiprocessor Systems. 2007 IEEE 13th International Symposium on High Performance Computer Architecture, 13–24.

[56] Zhenyuan Ruan, Shihang Li, Kaiyan Fan, Marcos K. Aguilera, Adam Belay, Seo Jin Park, and Malte Schwarzkopf. 2023. Unleashing True Utility Computing with Quicksand. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems* (Providence, RI, USA) *(HOTOS '23)*. Association for Computing Machinery, New York, NY, USA, 196–205. doi:10.1145/3593856.3595893

[57] Zhenyuan Ruan, Seo Jin Park, Marcos K. Aguilera, Adam Belay, and Malte Schwarzkopf. 2023. Nu: Achieving Microsecond-Scale Resource Fungibility with Logical Processes. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 1409–1427. https://www.usenix.org/conference/nsdi23/presentation/ruan

[58] Andy Rudoff, Chet Douglas, and Tiffany Kasanicky. 2021. Persistent Memory in CXL. https://www.snia.org/sites/default/files/PMSummit/2021/snia-pm-cs-summit-Rudoff-PM-in-CXL-2021.pdf SNIA Persistent Memory and Computational Storage Summit.

[59] Dirk Schmidl, Christian Terboven, Andreas Wolf, Dieter an Mey, and Christian Bischof. 2010. How to Scale Nested OpenMP Applications on the ScaleMP vSMP Architecture. In *2010 IEEE International Conference on Cluster Computing*. 29–37. doi:10.1109/CLUSTER.2010.38

[60] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 69–87. https://www.usenix.org/conference/osdi18/presentation/shan

[61] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: A Fast Capability System. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles* (Charleston, South Carolina, USA) *(SOSP '99)*. Association for Computing Machinery, 170–185.

[62] Anton Shilov. 2025. Samsung, MemVerge, and H3 build 2TB CXL Memory Pool. https://www.anandtech.com/show/20009/samsung_-memverge_-and-h3-build-2tb-cxl-memory-pool.

[63] Xiang Song, Haibo Chen, Rong Chen, Yuanxuan Wang, and Binyu Zang. 2011. A case for scaling applications to many-core with OS clustering. In *Proceedings of the Sixth Conference on Computer Systems* (Salzburg, Austria) *(EuroSys '11)*. Association for Computing Machinery, New York, NY, USA, 61–76. doi:10.1145/1966445.1966452

[64] Akshay Srivatsan, Yuhan Deng, Katherine Mohr, Emma Sudo, Sebastian Ingino, Francis Chua, and Keith Winstein. 2026. Continuation-Centric Computing with Arca. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1989–2005. https://www.usenix.org/conference/osdi26/presentation/srivatsan

[65] Storage Networking Industry Association. 2024. Swordfish NVMe Model Overview and Mapping Guide. https://www.snia.org/sites/default/files/technical-work/swordfish/release/v1.2.6/pdf/Swordfish_v1.2.6_NVMeMappingGuide.pdf Version 1.2.6.

[66] Michael M. Swift, Muthukaruppan Annamalai, Brian N. Bershad, and Henry M. Levy. 2006. Recovering device drivers. *ACM Transactions on Computer Systems* 24, 4 (2006), 333–360.

[67] Ariel Szekely, Adam Belay, Robert Morris, and M. Frans Kaashoek. 2024. Unifying Serverless and Microservice Tasks with SigmaOS. In *Proceedings of the 30th Symposium on Operating Systems Principles*.

ACM, 385–402. doi:10.1145/3694715.3695947

[68] Bruce Walker, Gerald Popek, Robert English, Charles Kline, and Greg Thiel. 1983. The LOCUS distributed operating system. In *Proceedings of the Ninth ACM Symposium on Operating Systems Principles* (Bretton Woods, New Hampshire, USA) *(SOSP '83)*. Association for Computing Machinery, New York, NY, USA, 49–70. doi:10.1145/800217.806615

[69] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed Shared Memory with In-Network Cache Coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 277–292. https://www.usenix.org/conference/fast21/presentation/wang

[70] Zhao Wang, Yiqi Chen, Cong Li, Yijin Guan, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. CTXNL: A Software-Hardware Co-designed Solution for Efficient CXL-Based Transaction Processing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 192–209. doi:10.1145/3676641.3716244

[71] Zhao Wang, Yiqi Chen, Cong Li, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. Enabling Efficient Transaction Processing on CXL-Based Memory Sharing. In *the ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[72] Xingda Wei, Haotian Wang, Tianxia Wang, Rong Chen, Jinyu Gu, Pengfei Zuo, and Haibo Chen. 2023. Transactional Indexes on (RDMA or CXL-based) Disaggregated Memory with Repairable Transaction. arXiv:2308.02501 [cs.DB] https://arxiv.org/abs/2308.02501

[73] David Wentzlaff, Charles Gruenwald, Nathan Beckmann, Kevin Modzelewski, Adam Belay, Lamia Youseff, Jason Miller, and Anant Agarwal. 2010. An operating system for multicore and clouds: mechanisms and implementation *(SoCC '10)*. Association for Computing Machinery, New York, NY, USA, 3–14. doi:10.1145/1807128.1807132

[74] Emmett Witchel. 2024. Challenges and Opportunities for Systems Using CXL Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 2. doi:10.1145/3620666.3655590

[75] Lars Wrenger, Florian Rommel, Alexander Halbuer, Christian Dietrich, and Daniel Lohmann. 2023. LLFree: Scalable and Optionally-Persistent Page-Frame Allocation. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 897–914. https://www.usenix.org/conference/atc23/presentation/wrenger

[76] Fangnuo Wu, Mingkai Dong, Gequan Mo, and Haibo Chen. 2023. TreeSLS: A Whole-system Persistent Microkernel with Tree-structured State Checkpoint on NVM. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3600006.3613160

[77] Yi Xu, Suyash Mahar, Ziheng Liu, Mingyao Shen, and Steven Swanson. 2024. CXL Shared Memory Programming: Barely Distributed and Almost Persistent. arXiv:2405.19626 [cs.DC] https://arxiv.org/abs/2405.19626

[78] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, and Michael Stonebraker. 2014. Staring into the abyss: an evaluation of concurrency control with one thousand cores. *Proc. VLDB Endow.* 8, 3 (nov 2014), 209–220. doi:10.14778/2735508.2735511

[79] Gerd Zellweger, Simon Gerber, Kornilios Kourtis, and Timothy Roscoe. 2014. Decoupling Cores, Kernels, and Operating Systems. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation* (Broomfield, CO) *(OSDI '14)*. USENIX Association, USA, 17–31.

[80] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674. doi:10.1145/3600006.3613135

[81] Xu Zhang, Ke Liu, Yuan Hui, Xiaolong Zheng, Yisong Chang, Yizhou Shan, Guanghui Zhang, Ke Zhang, Yungang Bao, Mingyu Chen, and Chenxi Wang. 2025. DRack: a CXL-disaggregated rack architecture to boost inter-rack communication. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) *(USENIX ATC '25)*. USENIX Association, USA, Article 74, 19 pages.

[82] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Antonis Psistakis, Joshua Fried, and Asaf Cidon. 2025. My CXL Pool Obviates Your PCIe Switch. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems* (Banff, AB, Canada) *(HotOS '25)*. Association for Computing Machinery, New York, NY, USA, 58–66. doi:10.1145/3713082.3730393

[83] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, and Xiaosong Ma. 2016. Gemini: A Computation-Centric Distributed Graph Processing System. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 301–316. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/zhu

[84] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. 2024. Lupin: Tolerating Partial Failures in a CXL Pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems* (Austin, TX, USA) *(DIMES '24)*. Association for Computing Machinery, New York, NY, USA, 41–50. doi:10.1145/3698783.3699377

A Appendix Overview

This appendix provides additional implementation and recovery details for StarfishOS:

- Appendix B explains how the kernel modules other than memory allocators (i.e., IPC, scheduling, and notification) recover from partial failures.
- Appendix C explains how the memory allocators (i.e., the lock-free buddy system and the slab allocator) recover from partial failures.
- Appendix D explains filesystem request routing, client reconnection after instance recovery, and special filesystem operations.

Note that this appendix is not peer reviewed.

B IPC/Scheduler/Notifier Recovery

This section presents recovery mechanisms for three kernel modules of StarfishOS. These modules—cross-machine IPC, thread scheduling, and event notification—all use *Multi-Producer Single-Consumer (MPSC) queues* for coordination. To tolerate partial failures, StarfishOS introduces `DurableQueue`, a lock-free, crash-consistent MPSC queue.

We first present the basic design of `DurableQueue` in §B.1, including its data structure, enqueue/dequeue algorithms, and recovery mechanism. We then describe its detailed designs for each kernel module in §B.2.

B.1 Basic Design of `DurableQueue`

`DurableQueue` is (1) lock-free, preventing failed machines from leaving locks held, and (2) crash-consistent, ensuring that the queue remains recoverable after failures. Its design builds on the persistent queue proposed by Friedman et al. [33]. StarfishOS extends its data structure and develops specialized enqueue and dequeue algorithms to meet the requirements of different kernel modules.

Data Structure. `DurableQueue`'s nodes embed a `status` and a `payload` defined by each specific kernel module.

Unified Data Structure for StarfishOS Queue.

```
Node {
Node* next;
char status ∈ {INIT, DOING, DONE, ABORT};
Payload payload; /* Defined by specific use cases
(IPC/Sched/Noti) */
}
Queue { Node* head; Node* tail; }
```

Enqueue (Algorithm 1). The `enq()` operation can be called by multiple producers (one case is when many clients want to send IPC to one server). Each producer:

1. Allocates a node with `status = INIT`, embeds the request in the `payload`, and `FLUSH(node)` (Line 1).
2. Links it into the list with `CAS` (Line 6).
3. Flushes the cacheline of the linking pointer (Line 7).
4. Changes `tail` to point to the new node (Line 8).

The **linearization point** of the `enq()` operation is a successful `CAS` (Line 6 in Algorithm 1).

The algorithm **avoids blocking**: one crashed `enq()` does not block other `enq()` operations. This is because `enq()` is lock-free and if a producer crashes in the middle of `enq()`, other producers can help finish the linking (Lines 12–13), leaving the queue always in a consistent state.

Algorithm 1: `enq(payload)` for StarfishOS.

```
1  node ← new Node(INIT, payload); FLUSH(node)
2  while true do
3      last ← tail; next ← last->next
4      if last == tail then
5          if next == NULL then
6              if CAS(&last->next, next, node) then
7                  FLUSH(&last->next)
8                  CAS(&tail, last, node)
9                  return node
10         else
11             /* Help crashed enqueuer */
12             FLUSH(&last->next)
13             CAS(&tail, last, next)
```

Dequeue (Algorithm 2). The `deq()` operation is executed exclusively by one consumer (for IPC, the consumer is one single server thread). Once there are new nodes enqueued to the queue, the consumer:

1. Dequeues the node (Line 9 in Algorithm 2).
2. Changes the node's status from `INIT` to `DOING` (Line 10).
3. Processes the payload via a use-case-specific `handle()` function (Line 12).
4. After `handle()` succeeds, attempts to change the status from `DOING` to `DONE` with a `CAS` (Line 13) and persists the status cacheline (Line 14).

Note that Line7 in Algorithm 2 is the same as Line12–13 in Algorithm 1, which is used to fix the inconsistency caused by enqueuer crashes.

Recovery. The `status` field serves as queue-level recovery metadata that records how far a request has progressed and guides post-crash handling. Under a producer crash, recovery determines whether the request was ever published to the queue; under a consumer crash, recovery determines whether handling had not started, completed, or stopped in an inconsistent intermediate state.

- **Producer crash:** a producer crash causes no blocking issues since `enq()` is lock-free and always leaves the queue in a consistent state (discussed in *Enqueue* paragraph). For the node that the producer is enqueuing, there are two cases: (1) if the producer fails before the linearization point, the request is absent; (2) if the producer fails after the linearization point, the request is committed and will eventually be handled by the consumer. After recovery,

Algorithm 2: `deq()` for StarfishOS.

```
1  while true do
2      first ← head; last ← tail; next ←
         first->next
3      if first == head then
4          if first == last then
5              if next == NULL then
6                  return NULL
7              FLUSH(&last->next); CAS(&tail, last,
                 next)
8          else
9              n ← next
10             if CAS(&n->status, INIT, DOING) then
11                 FLUSH(&n->status)
12                 handle(n) /* Varies by specific use case */
13                 CAS(&n->status, DOING, DONE)
14                 FLUSH(&n->status)
15             CAS(&head, first, next)
16             return
```

the producer can determine whether to retry the absent request or not.

- **Consumer crash:** a consumer crash causes no blocking: producers can continue enqueuing new nodes, and there is no other consumer to block. However, it may leave some nodes in intermediate states. StarfishOS recovers these nodes according to their `status`: (1) `INIT` means the request was published but the consumer failed before starting to handle it; the recovery procedure can simply retry the request; (2) `DOING` means the request may have stopped after partial execution, recovery can mark it as `ABORT` to notify the producer, or fix inconsistent state through system service state recovery (detailed in each use case); (3) `DONE` means handling completed and the response is valid, recovery will skip it; (4) `ABORT` records an explicit canceled request, the recovery will skip it.

B.2 DurableQueue for IPC/Scheduler/Notifier

The section shows detailed designs of DurableQueue for three core OS components: IPC, Scheduler, and Notifier.

B.2.1 IPC Queue. For inter-machine IPC, the queue serves as a message channel. The producer allocates a variable-sized node to accommodate the `msgBuffer`. After enqueuing, the producer spins on `n->status` to wait for completion of the request. Once the consumer executes the message and successfully changes `status` from `DOING` to `DONE`, the producer reads the response from `msgBuffer`. The consumer handles the message based on the `msgType`, it writes the response back to `msgBuffer`, and persists the `msgBuffer` during `handle(n)`.

- **Payload:** `msgType`, `msgBuffer` (variable-sized).
- **Queue:** `IPCQueue ipcQueue[MAX_MACHINE_NUM]`.
- **Producer:** applications or system services that invoke IPC.
- **Consumer:** the target system service instance's thread, which is invoked by the interrupt-and-polling mechanism.
- **Producer crash:** As discussed in §B.1, a producer crash before/after the linearization point of `enq()` will leave the IPC request absent/committed, respectively. For IPC, after application-level recovery, the application can check the `status` of its previous in-flight IPC requests to determine whether each request was committed and retry it if necessary.
- **Consumer crash:** A consumer (i.e., a system-service server) crash may leave a request in an intermediate state. During recovery, the replacement server handles the request according to its node's `status`: (1) for `INIT`, it retries the request because processing has not started; (2) for `DOING`, it consults service-specific recovery metadata to determine whether the request took effect. For example, a recovering filesystem instance checks its p-log. If the corresponding operation is recorded as completed, the recovered filesystem state already includes its effects, so the server attempts to change the node's status from `DOING` to `DONE`. If the p-log contains no record of the corresponding operation, the request has not affected the filesystem state, so the server can safely re-execute it. (3) For `DONE` or `ABORT`, it skips the request because it has already reached a terminal state.

B.2.2 Scheduler Queue. Each CPU core has a scheduler queue (`SchedQueue`) on CXL, containing threads (`Thread*`) to be migrated to that CPU. StarfishOS maintains a global array of these queues (`SchedQueue schedQueues[MAX_CPU_NUM]`). When an operation such as `sched_setaffinity()` migrates a thread to a remote CPU, it enqueues a node carrying the thread's `Thread*` into the target CPU's `SchedQueue`. During each scheduling tick, the target CPU polls its `SchedQueue`, and calls `handle(n)` to dequeue all threads (`Thread*`) in the queue and inserts them into the CPU's local volatile scheduler queue. The target CPU then selects a thread from its local volatile runqueue for execution; migrated threads are scheduled normally once inserted into this queue.

- **Payload:** `Thread*`.
- **Queue:** `SchedQueue schedQueues[MAX_CPU_NUM]`.
- **Producer:** application calling `sched_setaffinity()` or scheduler deciding to migrate a thread to a remote CPU.
- **Consumer:** the target CPU core's scheduler.
- **Producer crash:** After a producer crash, the shared queue itself remains functional and other machines can still enqueue threads to that CPU. In-flight scheduling requests may be lost, but this requires no recovery: StarfishOS treats the thread as never having migrated, and the thread fails along with its source machine.

- **Consumer crash:** A consumer crash means that the consumer's machine has failed, causing the state of all private queues on that machine to be lost. After the machine recovers, each CPU's scheduler reconstructs its private queue and re-enqueues any interrupted `DOING` nodes from the CPU's `SchedQueue`. CPUs on other machines remain unaffected because each CPU has its own shared queue.

B.2.3 Notification Queue. Notifications are synchronization objects (e.g., semaphores, condition variables) that allow a thread to wait for an event and another thread to wake it when the event occurs. Each notification maintains a shared waiting list (`NotiQueue`) on CXL, and is associated with multiple waiting threads plus one notifying thread. A waiting thread enqueues its `Thread*` and the ID of the CPU on which it is currently running (`cpu_id`) into the `NotiQueue`, while a notifying thread dequeues the node and wakes the thread on the CPU identified by `cpu_id`.

- **Payload:** `Thread*` (waiting thread), `cpu_id` (CPU ID that the waiting thread is currently running on).
- **Queue:** One `NotiQueue` for each notification.
- **Producer:** multiple waiting threads.
- **Consumer:** one notifying thread.
- **Producer crash:** Under a producer crash, the shared queue remains functional, allowing other threads to continue waiting or notifying. An in-flight wait request may be lost, but this requires no recovery because the waiting thread fails along with its machine.
- **Consumer crash:** A consumer crash means that the notifying thread's machine has failed. A crash during `deq()` does not corrupt the `NotiQueue`; it only leaves pending wake-up requests without a consumer. If the crash occurs during `handle(n)`, which schedules the waiting thread on a local or remote CPU, the interrupted scheduling operation is handled by the `SchedQueue` failure guarantees described above. The main consequence is therefore that some waiting threads may not be woken. This requires no additional kernel recovery because StarfishOS treats the entire application containing the failed notifying thread as failed; the application must instead be restarted at the application level.

C Memory Allocator Recovery

To tolerate partial failures in CXL memory allocation, StarfishOS extends its original kernel-space buddy/slab allocators. Specifically, StarfishOS replaces the original buddy allocator with an LLFree [75]-based buddy system and enhances the slab allocator with a recovery mechanism.

In this section, we discuss the following topics:

- **Background on LLFree (§C.1):** explains LLFree's data structure, allocation/free, and recovery mechanism.
- **Our Buddy System Recovery (§C.2):** explains our recovery mechanism for LLFree-based buddy system.
- **Slab System Recovery (§C.3):** explains our recovery mechanism for the slab system.

C.1 Background on LLFree

LLFree [75] is a lock-free, page-level allocator that supports allocations of size $2^o \times P$, where $o \in \{0, \ldots, 10\}$ and $P = 4\,\text{KiB}$, matching StarfishOS's original buddy system. StarfishOS uses only 4 KiB base pages and 2 MiB huge pages, for which LLFree provides crash consistency. This is sufficient for StarfishOS because:

- User-space allocators, such as libc's malloc and rpmalloc, obtain 2 MiB regions from the kernel and manage smaller objects internally.
- In the kernel-space, the slab allocator handles sub-4 KiB objects, while LLFree handles page-level allocations.

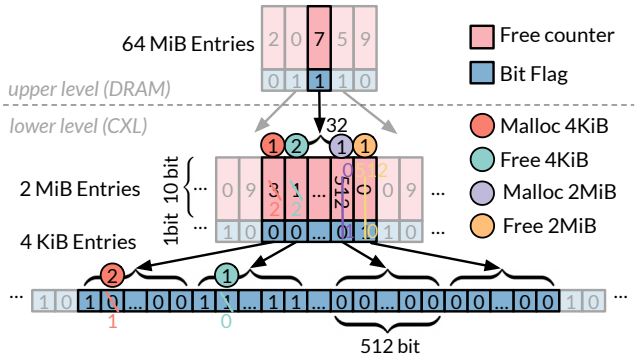


Figure 19. LLFree Allocator

C.1.1 Memory Organization. As shown in Figure 19, LLFree organizes the memory into two levels:

(1) Upper Level: The upper level summarizes the free capacity of each 64 MiB region and allows each CPU to choose a region for allocation (detailed in the original paper [75]).

(2) Lower Level: After the upper level selects a 64 MiB region, the lower level performs allocations and records the exact allocation state at two granularities:

- **2 MiB entries**: Each entry represents a 2 MiB region and contains a free counter c_L (records the number of free 4 KiB pages in the region) and an allocation flag a (indicates whether the entire region is allocated as a huge page).
- **4 KiB entries**: Each 2 MiB region has a 512-bit bitmap, with one bit per 4 KiB page: 0 denotes free and 1 denotes allocated.

Only the lower-level metadata must persist for crash recovery; the upper level can be reconstructed from it. Thus, StarfishOS places the lower-level metadata in CXL, and upper-level in DRAM.

C.1.2 4 KiB Base Pages.

Allocation. LLFree (1) atomically decrements c_L, (2) scans the bitfield for a free 4 KiB page, and (3) atomically changes its bit from 0 to 1.

Free. LLFree (1) atomically clears the page's bit from 1 to 0, and (2) atomically increments c_L.

Crash consistency. Each metadata update is atomic, but a crash between the two updates may leave c_L underestimating the available space and cause a temporary false out-of-memory result. The bitfield remains the ground truth, so the mismatch cannot cause double allocation or double free; recovery reconstructs c_L from the bitfield.

C.1.3 2 MiB Huge Pages.

Allocation. LLFree allocates a 2 MiB huge page with one CAS on its metadata entry:

$$(c_L = 512, a = 0) \longrightarrow (c_L = 0, a = 1).$$

Free. Freeing the huge page uses one CAS for the reverse transition:

$$(c_L = 0, a = 1) \longrightarrow (c_L = 512, a = 0).$$

Crash consistency. Because each operation is a single CAS, a crash leaves the entry either unchanged or fully updated; no intermediate metadata state is exposed.

C.1.4 LLFree's Recovery Procedure.

LLFree assumes a stop-the-world failure, so no other allocations or frees run concurrently during recovery. Specifically, the allocator reconstructs metadata as follows:

1. Scan the bitfield of each 2 MiB entry.
2. Recompute c_L as the number of zero bits.
3. If c_L is different from the original c_L, CAS the 2 MiB entry to use the correct c_L.

C.2 StarfishOS's Buddy System Recovery

Challenges. The original LLFree recovery assumes a stop-the-world failure. However, this assumption does not hold in StarfishOS. In StarfishOS, when one physical machine crashes, surviving machines continue accessing the CXL allocator. This means that recovery cannot assume that each 2 MiB entry stays unchanged after it is first read.

Solution. StarfishOS extends LLFree with region-scoped quiescence: it temporarily stops allocation and free operations only on the 64 MiB regions being repaired, while other regions remain accessible. Recovery proceeds as follows:

1. **Block:** Publish a recovery flag for the affected 64 MiB regions. New allocation and free operations must wait or retry elsewhere. To drain in-flight operations, the recovery thread sends a quiescence request to every live machine, which acknowledges after its existing operations on these regions finish.
2. **Recover:** After receiving all acknowledgments, scan each 2 MiB entry's bitmap and repair its counter without concurrent modifications.
3. **Resume:** Clear the recovery flag and resume allocation and free operations on the repaired regions.

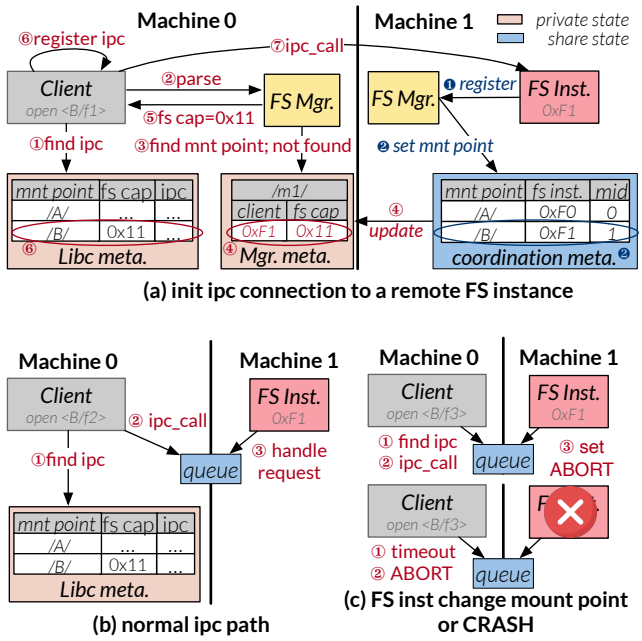


Figure 20. Filesystem service details.

C.3 StarfishOS's Slab Recovery

Buddy system manages page-granularity CXL allocations, while smaller kernel objects are allocated from per-CPU slab instances built on top of these pages. Although each slab instance remains lock-based, its lock is private to one CPU and therefore does not block other slab instances. During recovery, StarfishOS reconstructs the locks of the affected slab instances.

Still, we need to ensure that interrupted allocations/frees leave the slab instance in a consistent state. StarfishOS therefore adds a small undo record to each slab header.

Per-slab undo record. Each slab allocator stores the free-list head (`free_list_head`), the number of free objects (`free_cnt`), and the lock and list links used by the allocator. StarfishOS extends it with an undo record containing:

- the operation type (`Alloc`, `Free`, or `None`);
- the previous free-list head; and
- the previous free-object count.

Before modifying the slab metadata, the allocator writes the undo record to CXL and persists it with a flush and fence. Because each slab serializes operations with its lock, it maintains only one undo-log entry for the current in-flight allocation or free. Once the operation completes and its updates are persisted, the entry can be reused by the next operation.

D Details of Filesystem Services

This section details how clients communicate with filesystem instances and refresh their routing information after an instance failure or reassignment.

Filesystem instance registration (Figure 20 (a)). When a filesystem instance starts, it registers the mount point that it serves in the *coordination metadata* (❶, ❷). The *coordination metadata* records which filesystem instance currently owns each mount point (the bottom-right box in Figure 20 (a)). Each entry identifies the mount point, its current filesystem instance, the hosting machine, and a mapping version.

Connection establishment (Figure 20 (a)). When a client on Machine0 opens `</B/f1>` (①), if the path is not valid locally, the client connects to the local filesystem manager (②) to resolve the path. If the local filesystem manager cannot resolve `/B` locally (③), the manager consults the shared coordination metadata to locate the filesystem instance responsible for that mount point (④). After the manager returns the target instance information (⑤), the client sets up the IPC connection (⑥) and communicates directly with the remote filesystem instance on Machine1 (⑦). The IPC connection is backed by a FIFO request queue in CXL, consistent with how remote IPC is implemented elsewhere in StarfishOS.

Cached request routing (Figure 20 (b)). After the IPC connection is established, subsequent operations on the same mount point can reuse the cached routing information without consulting the manager again.

Failure recovery and mapping refresh (Figure 20 (c)). As shown in main text, when a filesystem instance fails, StarfishOS starts a replacement instance on a surviving machine and reconstructs its state from SSD and the failed instance's p-log. Then, clients can reconnect to the replacement instance and retry the requests. The detailed recovery process is as follows:

1. Once detected the failure of a machine, the failure detector marks all IPC connections to services on the failed machine as `ABORT`, notifying clients that its requests cannot be completed.
2. StarfishOS then starts a replacement instance on a surviving machine and reconstructs its state from SSD and p-log.
3. After state recovery completes, StarfishOS updates the mount-point entry in the coordination metadata to identify the replacement instance and increments the entry's version.
4. A client observing `ABORT` discards its cached mapping and old IPC connection, queries the coordination metadata, establishes a new connection to the replacement instance, and retries the request when appropriate.

Support for mmap(), rename() and symbolic links. Remote `mmap()` requests are handled by allocating a CXL memory region, putting the file's data in it, and mapping the region to the client's virtual address space. Currently, the `rename()` operation is not supported across mount points. Symbolic links are supported since they are represented as path strings. Pathname resolution is performed on the client side, so a symbolic link that crosses mount points can be resolved by consulting the coordination metadata again when the lookup traverses into another filesystem instance.